

Fin 350 - Financial Markets

Project Report

**As a partial fulfillment of the Financial Markets
course**

By

Amine Bessaâd

Yassine Essaâdi

Mohamed Adnen Guirat

TunisieValeurs
LA MAISON DE L'ÉPARGNANT

ACADEMIC SUPERVISOR

Dr. Eymen ERRAIES

PROFESSIONAL SUPERVISOR

Mr. Ramzi JERBI

Academic Year

2025-2026

Table of Contents

Introduction

Step 1 & Step 2 — Broker Selection and Account Setup

Step 3

2.1. Macroeconomic Analysis

2.2. Macroeconomic Sector Analysis and Cycle Positioning

2.3. Linking Economic and Financial Perspectives

2.4. Sectoral Classification and 20 Selected Companies Justification

Step 4 — Adjusted Returns

Step 5 — Shortlisting the 10 Stocks

Step 6 — CAPM and Portfolio Optimization

5.1. Data Collection and Alignment

5.2. CAPM Calculations

5.3. Markowitz Optimal Portfolio

5.4. Portfolio Risk–Return Metrics

5.5. Deliverables

Step 7 — Market Entry Strategy and Order Execution

6.1. Technical Analysis Indicators (MACD, RSI, OBV, Volume)

6.2. Cash Management as a Risk Control Tool

6.3. Entry Point Determination

6.4. Intraday Monitoring and Trade Adjustments

6.5. Execution Summary

6.6. Budget Allocation

Step 8 — Portfolio Rebalancing Decisions

Step 9 — Performance Evaluation and Return Calculations

Ending Notes

Step 10 — Appendix Python Technical Analysis Code and Data Sources

Introduction

This report documents the comprehensive investment strategy and execution process undertaken for the Financial Markets project course (**FIN 350**), under the instruction of **Dr. Eymen Errais**. The project simulates the experience of a junior trader, tasked with constructing and managing an equity portfolio listed on the Tunis Stock Exchange (TSE) with the primary objective of maximizing risk-adjusted returns over a three-month horizon.

The project mandates the application of core financial theories and analytical techniques acquired throughout the program. Our methodology follows a structured, multi-stage investment process. This begins with the critical selection of a reputable stockbroker and the negotiation of favorable terms, establishing the foundation for operational efficiency. A rigorous selection process was then employed to identify a universe of promising equities, which was subsequently narrowed down through fundamental financial statement analysis.

The core of the strategy involves the application of modern portfolio theory to construct an optimized portfolio tailored to a defined risk profile. Execution tactics, informed by technical analysis, were used to determine optimal market entry points. Finally, the performance of the implemented strategy is compared with relevant benchmarks using standardized performance and risk metrics.

This report methodically outlines each phase of this project, from initial setup and security selection to final performance evaluation, providing a complete overview of the analytical thoroughness and strategic decision-making applied throughout the project.

Step 1 & Step 2 — Broker Selection and Account Setup

1. Broker Selection

The selection of an appropriate brokerage partner was the pillar-stone in this simulation; critical for ensuring operational efficiency, access to reliable market data, and adherence to ethical standards. After a comprehensive review of prominent brokers operating in the Tunisian financial market, our group selected Tunisie Valeurs as our intermediary.

Our decision was based on the following key criteria:

- **Reputation and Ethical Standing:** **Tunisie Valeurs** is a leading financial intermediary in Tunisia, renowned for its strong reputation, longevity in the market, and strict adherence to the regulations set forth by **the Financial Market Council (CMF)**. This ensures that all trading activities are conducted within a secure and professionally governed framework, aligning with the project's emphasis on ethical standards.

- **Quality of Trading Platform:** The broker provides a modern, intuitive, and stable online trading platform. It offers robust charting tools, real-time data streams, and efficient order execution capabilities, which are indispensable for conducting the technical and fundamental analysis required by our strategy.
- **Research and Analytical Support:** Tunisie Valeurs distinguishes itself by providing clients with valuable market research, analysis, and periodic reports. Access to this in-house expertise provides a significant advantage for making informed investment decisions, a feature not all local brokers offer to the same extent.
- **Local Presence:** The availability of a local branch provides a crucial point of contact for personalized service, facilitates the swift resolution of potential administrative issues, and allows for in-person consultation, which we deem essential for a seamless client experience.
- **Competitive Fee Structure:** While the initial published fee schedule was a consideration, Tunisie Valeurs's demonstrated willingness to negotiate terms based on trading volume made them a competitive and attractive choice for this active management simulation.

In conclusion, Tunisie Valeurs was selected for its combination of professional integrity, an extensive and accessible local network, technological capability, and value-added services. This blend of physical and digital resources provides the optimal environment for executing a sophisticated trading strategy while mitigating the logistical challenges of our geographic dispersion.

2. Negotiation of Fees

A fundamental principle of portfolio management is the vigilant control of transaction costs, as they can substantially diminish net returns over time. Recognizing this, we proactively engaged with Tunisie Valeurs to negotiate favorable terms, leveraging our position as finance students from a prestigious institution.

Our negotiation was not solely based on standard arguments but was significantly strengthened by our academic profile:

- **Institutional Reputation and Future Potential:** We presented ourselves as a group of ambitious TBS students, an institution known for producing leaders in the finance

sector. We emphasized that this project was our first hands-on foray into the Tunisian capital markets. Our argument was that by facilitating our entry with competitive fees, Tunisie Valeurs would not only support academic development but also cultivate a relationship with future finance professionals who are likely to become long-term clients upon entering the workforce.

- **Pioneering Initiative and Goodwill:** As one of the first student groups to approach them for this specific academic project, we were met with a receptive and encouraging attitude. The brokers recognized the value of this initiative and were inclined to support our practical learning experience, viewing it as an investment in future market participants.

The representatives at Tunisie Valeurs were notably supportive of our academic pursuit and acknowledged the mutual benefit of engaging with TBS students. They granted us a privileged fee reduction as an incentive to encourage our entry into real-world financial trading.

Consequently, we successfully negotiated a reduction in the brokerage fee, from a standard rate of 0.6% down to 0.2% (20 basis points) for all equity transactions. This exceptional rate, evidenced in the account statement provided in Figure 1 below, provides a significant competitive advantage for our portfolio by minimizing frictional costs. This achievement underscores the practical value of our academic standing and provides a clear testament to our proactive approach and negotiation skills.

3 : REMUNERATION DE TUNISIE VALEURS

En rémunération de ses services TUNISIE VALEURS applique sur le Client les tarifs figurant ci-dessous :

-Frais de tenue de compte au taux annuel 0.200 % Hors Taxes avec un minimum de quarante (40) dinars Hors Taxes et un maximum de trois cents (300) dinars Hors Taxes, par an. Les frais de tenue de compte sont payables par semestre.

-Frais de courtage en cas de transactions, au taux de 0.20 % Hors Taxes.

Le Client accepte, sans réserve, ces tarifs.

Step 3 Macroeconomic Analysis:

1. Introduction

This section presents a structured macroeconomic analysis of the Tunisian economy, examining the fundamental drivers and challenges that have characterized its recent performance. By evaluating key indicators across core domains—including growth sustainability, fiscal and external balances, monetary stability, and labor market dynamics—we aim to construct a holistic picture of the nation's economic health. This assessment of Tunisia's macroeconomic fundamentals is critical for identifying both the potential opportunities and salient risks inherent in the market, thereby providing a robust evidence base to inform strategic trading and investment decisions.

2. Growth Dynamics: A Tale of Two Economies

The Q2 2025 data reveals a dual-track economy:

- **Real GDP Recovery:**

Real GDP growth accelerated to **3.16% YoY** in **Q2 2025**, the strongest since **2022**. This is driven by a powerful rebound in Agriculture (**+9.84% YoY**) and a recovery in Industries (**+3.97% YoY**) and Services (**+2.77% YoY**). This suggests a broad, though nascent, recovery.

- **Nominal GDP Illusion:**

However, nominal GDP growth is soaring at **+16.54% YoY**, far outstripping real growth. This indicates that the reported economic expansion is heavily inflated by price increases rather than genuine volume growth.

- **TSE Implication:**

This disparity is crucial. Companies reporting strong revenue growth may simply be benefiting from inflation. Investors must scrutinize volume growth and market share to identify truly healthy businesses. The strong agricultural performance could benefit listed agribusiness and food processing companies (e.g., SIAM), while the industrial recovery bodes well for manufacturing exporters.

3) The Investment Collapse: The Core Vulnerability

The most alarming data point is the precipitous decline in the Investment-to-GDP ratio, which fell to a critically low **7.9% in 2024** from a **pre-2019 average of 21-23%**. A ratio below **10%** indicates the capital stock is eroding, undermining future productivity and job creation.

- **TSE Implication:**

Negative: This signals a bleak medium-term outlook for sectors dependent on public spending and large-scale capital expenditure (e.g., construction, infrastructure, heavy industry). Listed companies in these sectors face a shrinking addressable market.

Opportunity: This may benefit companies that require low capital intensity or are focused on maintenance and operational efficiency rather than expansion.

4) External Sector: Deep Imbalances with a Silver

Lining

Tunisia's economy is highly open (Openness Ratio >100%), making it vulnerable to external shocks.

Record Trade Deficit: The net exports deficit widened to a massive **-22.9%** of GDP in **2024**, a significant drag on growth driven by high import dependence.

Improving Coverage Ratio: A positive trend is the rising Coverage Ratio (exports/imports), which reached **88.8% in 2024**. This indicates that export performance is improving relative to imports, a key metric for export-oriented listed companies.

Export Concentration Risk

The data reveals a critical vulnerability: Tunisia's exports are highly concentrated in a few European markets. Historically, **France, Italy, Germany, and Spain** consistently absorb **over 50% of all exports**, with the top 5 partners accounting for **65-76%** of the total.

Key Risk: This extreme dependence on European demand means any slowdown in the EU (e.g., from an energy crisis, recession, or high inflation) would immediately hit Tunisian corporate revenues and, by extension, TSE-listed exporters.

Diversification Trends: There are promising signs of diversification. Exports to China saw a sharp rise from **2017**, and the USA became a significant market, exceeding **\$1.1B by 2018**. However, exports to other regions like Sub-Saharan Africa remain negligible, representing an untapped opportunity.

TSE Implication: This is a key differentiator for stock selection.

Overweight Export Champions with Diversified Markets: Companies that are not only competitive exporters (benefiting from a weak dinar) but have also diversified beyond the EU into markets like the USA, China, or have potential in Africa are significantly more resilient. They are hedged against an EU slowdown.

Underweight EU-Dependent Exporters: Companies whose revenues are overwhelmingly tied to EU demand carry higher risk due to geographic concentration.

Underweight Import-Dependent Firms: Companies reliant on imported raw materials (e.g., energy, pharmaceuticals) will continue to face severe cost pressures and margin squeeze from the trade deficit and dinar weakness.

5) Labor Market: Structural Stagnation

Unemployment remains stubbornly high at **~15-16%**, with female unemployment above **20%**. This is a structural problem, indicating that even periods of growth do not create sufficient jobs.

TSE Implication: Weak household income and consumer demand will continue to challenge companies in the consumer discretionary sector (e.g., retail, durable goods).

Investors should favor companies in consumer staples (e.g., Poulina Group) that offer essential products with inelastic demand.

6) Inflation & Profitability: The CPI/PPI Squeeze

Inflation is moderating but remains elevated at **~5.5% in 2025**. The CPI-to-PPI ratio analysis is particularly insightful for corporate profitability.

2019-2022: The ratio was low (**<85%**), indicating producers (PPI) faced rising costs faster than they could pass them to consumers (CPI). This led to a severe profit margin squeeze.

2023-2025: The ratio has risen to **>92%**, suggesting firms are now successfully passing costs to consumers.

TSE Implication: The easing of the margin squeeze is a positive development for corporate earnings on the TSE. Companies with strong pricing power and brand loyalty are best positioned to protect and rebuild their margins in this environment. This trend could lead to upward revisions in earnings forecasts for resilient players.

7) Budget Deficit Analysis: The Missing Piece

While official websites did not contain explicit data, the budget deficit is a critical driver of Tunisia's macroeconomic context and directly impacts the TSE through crowding-out effects and sovereign risk.

Source of the Deficit: Tunisia has run persistent fiscal deficits for over a decade, driven by a large public wage bill, costly food and energy subsidies, and weak tax revenues. The COVID-19 pandemic and the response to the social crisis exacerbated this.

Current Situation: The high inflation (**16.54%** nominal GDP growth) and weak real growth (**3.16%**) suggest the deficit remains significant. The government's nominal tax revenues may be rising with inflation, but its expenditures are also soaring, particularly on subsidies and debt servicing (which is inflated by high interest rates and a weak dinar).

Impact on the Economy and TSE:

Crowding-Out Effect: To finance the deficit, the government must borrow heavily from the domestic market. This competes with the private sector for scarce capital, keeping interest rates high. This increases the cost of borrowing for listed companies, dampening their investment and expansion plans.

Sovereign Risk: A high deficit increases Tunisia's sovereign risk premium, leading to pressure on the Tunisian Dinar (TND) and making international borrowing more expensive. This contributes to the imported inflation problem.

Policy Constraints: The government has limited fiscal space to stimulate the economy or support key sectors, making a private-sector-led recovery essential.

TSE Implication: The precarious fiscal situation means that monetary policy will likely remain restrictive to anchor inflation and support the currency. This is a headwind for interest-rate-sensitive sectors like banking and real estate. However, banks with a strong base in government bond trading may benefit. Any sign of credible fiscal consolidation could be a major positive signal for the market.

Investment Conclusion & TSE Strategy:

Tunisia's macroeconomic landscape for 2025 is one of cautious optimism overshadowed by structural fragility. For investors on the Tunis Stock Exchange, this implies a defensive and selective strategy:

Overweight: Export-oriented companies (benefiting from dinar weakness and improving coverage ratios), companies with strong pricing power in consumer staples, and firms in sectors showing cyclical recovery (Agriculture, selective Industries).

Underweight: Companies heavily reliant on domestic capital investment, import-dependent businesses with weak pricing power, and consumer discretionary stocks vulnerable to weak household demand.

Key Monitorable: The single most important indicator to watch is the Investment-to-GDP ratio. A sustained recovery above 15% would be the clearest signal of a fundamental improvement in Tunisia's growth potential and a bullish indicator for the broader market.

Macroeconomic Sector Analysis and Cycle Positioning

1. Financial Services Sector — Late Recovery to Early Expansion

- Real GDP growth reached **3.16% YoY in Q2 2025**, providing a supportive macroeconomic backdrop for financial activity.
- Inflation remains elevated at approximately **5.5% in 2025**, improving nominal interest income and banking margins.
- Private-sector credit growth remains weak due to a critically low **investment-to-GDP ratio of 7.9% in 2024**, which continues to limit demand for productive loans.
- Unemployment remains structurally high at **15–16%**, constraining household borrowing capacity and insurance penetration, reflecting persistent labor market rigidities.
- Banking sector growth is therefore largely driven by sovereign financing and holdings of government securities, rather than private investment, a dynamic emphasized by both the Central Bank of Tunisia and the IMF in their analysis of banking sector balance sheets (**BCT, Rapport Annuel, 2024; IMF, Article IV Consultation, 2024**).

Interpretation:

The sector is moving out of recovery but has not yet entered a broad-based expansion, justifying a cautious and selective outlook.

2. Industrial Sector — Early Recovery

- Industrial value added increased by **3.97% YoY in Q2 2025**, signaling a gradual recovery after prolonged stagnation, according to sectoral GDP data released by the **INS (*Comptes Nationaux Trimestriels*, Q2 2025)**.
- Export-oriented manufacturing benefits from a weak Tunisian dinar, which has improved external competitiveness and supported export performance, particularly toward European markets.
- The recovery remains fragile due to:
 - High energy import dependence, which continues to expose industrial firms to elevated production costs (**BCT, *Rapport Annuel*, 2024**).
 - Persistently weak capital formation, with total investment remaining below **10% of GDP**, limiting capacity expansion and modernization
- Construction activity continues to lag, reflecting weak domestic demand and limited public infrastructure spending, as shown by recent industrial production and construction indicators (**INS, *Indices de la Production Industrielle*, 2024**).

Interpretation:

The recovery is externally driven and uneven, favoring export-oriented industrial firms over domestically focused or capital-intensive producers.

3. Consumer Goods Sector — Divergent Cycle Dynamics**a) Consumer Staples — Defensive Expansion**

- Demand for essential goods remains stable despite inflationary pressures, reflecting the inelastic nature of food and basic household consumption (**INS, *Indice des Prix à la Consommation***).
- High unemployment (>15%) and weak real income growth have limited discretionary spending but have not significantly affected staple consumption, according to labor market and household income data (**INS, *Enquête sur l'Emploi*, 2024**).
- Improved CPI-to-PPI pass-through since 2023 supports margin stabilization for essential goods producers, as firms have increasingly been able to transfer rising input costs to consumers.

b) Consumer Discretionary — Stagnation / Recession

- Elevated inflation and declining real purchasing power continue to suppress non-essential consumption, particularly for durable and semi-durable goods
- Weak credit growth further limits demand for durable goods, reflecting tighter lending conditions and subdued consumer confidence (**BCT, *Indicateurs Monétaires*, 2025**)

Interpretation:

The sector displays a clear split, with consumer staples showing resilience while discretionary segments remain under pressure

4. Agricultural Sector — Expansion

- Agricultural output grew by **9.84% YoY in Q2 2025**, representing the strongest performance among all major sectors.
- Growth is supported by:
 - Favorable climatic conditions
 - Improved agricultural yields
 - Rising export demand for agro-industrial products
- The sector contributes significantly to real GDP growth, particularly in interior regions where agriculture remains a key economic driver (**INS, *Indicateurs Régionaux de Développement*, 2024**).

Interpretation:

Despite strong cyclical momentum, the sector remains exposed to climate volatility and structural productivity constraints.

5. Energy Sector — Structural Contraction

- The energy sector continues to contract due to declining domestic oil and gas production, a trend consistently highlighted in Tunisia's energy and production statistics (**Banque Centrale de Tunisie, *Rapport Annuel*, 2024**).
- Rising energy import dependence has contributed to a widening trade deficit of -- **22.9% of GDP in 2024**, as shown by foreign trade and balance of payments data (**INS, *Commerce Extérieur*; BCT, *Balance des Paiements*, 2024**).
- Energy subsidies place additional pressure on public finances and limit fiscal space, reinforcing fiscal vulnerabilities identified by international institutions (**IMF, *Energy Subsidy Reform in Tunisia*, 2024**).
- High energy costs negatively affect industrial competitiveness and corporate margins, generating adverse spillovers across energy-intensive sectors.

Interpretation:

The sector represents a structural drag on economic performance, with any recovery dependent on long-term energy transition policies.

Linking Economic and Financial Perspectives

To ensure consistency between the macroeconomic assessment of Tunisia's economy and the financial selection of stocks on the Tunis Stock Exchange (TSE), an explicit link was established between the economic and financial perspectives. This approach allowed macroeconomic signals to be translated into sector allocation decisions and, ultimately, into a coherent stock selection strategy.

Role of Each Perspective

- **Macroeconomic Analysis (INS, Central Bank data):**
The macroeconomic analysis focused on identifying the key structural and cyclical features of the Tunisian economy, including the divergence between real and nominal GDP growth, the collapse in the investment-to-GDP ratio, persistent inflation, external imbalances, and labor market rigidity. These indicators were used to **identify sectors likely to outperform or underperform** in the short term, guiding the relative weighting of consumer staples, export-oriented industries, financial institutions, and capital-intensive sectors.
- **Financial and Market Analysis (TSE data, trading activity, technical indicators):**

The financial analysis focused on selecting stocks within the preferred sectors based on **market capitalization, average trading volume, and short-term price dynamics**. This ensured that the chosen securities were sufficiently liquid, tradable, and compatible with a 3-month investment horizon, while also allowing for effective risk management and portfolio rebalancing.

Note:

Our selection was narrowed from 80 listed stocks down to a focused portfolio of 23, based on three core criteria: liquidity, technical analysis, fundamental analysis and overall performance.

While liquidity was the primary filter to ensure tradability, we sought stocks that balanced all these aspects.

Industrials & Components

- **ONE TECH HOLDING** : The top-ranked stock overall. Selected as the model's highest-conviction pick because it optimally combines high liquidity (**Avg. Vol: ~26,172**) with strong fundamentals (**Mkt Cap: ~715.6M TND**), making it a core growth and stability anchor for the entire portfolio.

Consumer Goods

- **POULINA GROUP HOLDING**: A sector giant and top performer. Its massive fundamental scale (**Mkt Cap: ~2.65B TND**) and robust liquidity (**Avg. Vol: ~41,403**)

provide foundational stability and defensive characteristics, justifying its place as a primary portfolio cornerstone.

- **EURO-CYCLES** was chosen primarily for its strong technical signal. Despite a moderate **market cap (~129M TND)** and decent liquidity (**~7,444 avg vol**), it showed a clear breakout from a descending wedge pattern on high volume—a reliable bullish indicator that outweighed its fundamental profile for alpha potential.

Banks

- **BANQUE ATTIJARI DE TUNISIE**: Our optimal banking choice. It balances strong fundamentals (**Mkt Cap: ~2.61B TND**), reliable liquidity (**Avg. Vol: ~5,807**), and a superior risk-return profile. The model selected it over other banks (like BIAT or ATB) because this specific combination contributed more to overall portfolio efficiency.

Distribution

- **SMART TUNISIE**: It has respectable fundamentals (**~173 million TND market cap**) and good liquidity (**~4,861 avg. volume**). Technically, it exhibited a clean retest of a breakout level. After a strong initial rally, the price pulled back to the exact point of its prior breakout, held firmly, and began advancing again on renewed volume—a textbook sign of a healthy, continuing trend.

Construction & Materials

- **CARTHAGE CEMENT** : The undisputed liquidity anchor. Possesses the highest single-stock liquidity in our analysis (**Avg. Vol: ~108,900**) and sector dominance (**Mkt Cap: ~650.1M TND**). While cyclical, it was selected for its immense tradability (ensuring low trading costs) and its role in providing exposure to infrastructure and construction momentum.
- **ESSOUKNA** was selected as a high-conviction cyclical play. While it has a small market cap (**~11.9M TND**) and only moderate liquidity (**~3,351 avg vol**), it demonstrated strong performance momentum. The stock broke out of a multi-year base on surging volume, signaling a powerful cyclical upturn and a high-reward opportunity that justified its inclusion despite its smaller fundamental size

Insurance

MAGHREBIA VIE: was chosen for its strong fundamentals (**~197.5M TND cap**) and a promising technical setup. Despite very low liquidity (**~27 avg vol**), the stock formed a tight coiling pattern near its highs, signaling an imminent breakout. This technical potential justified accepting the low trading volume.

Health:

UNIMED: The clear leader in the Health sector, chosen for its dominant fundamentals (**~262.4 million TND market cap**) and exceptional liquidity (**~36,948 avg. volume**), making it both stable and easy to trade. Technically, it was in a strong, steady uptrend characterized by a series of higher highs and higher lows, consistently supported by its 50-day moving average. This combination of defensive sector leadership, high liquidity, and a clear bullish trend made it a mandatory portfolio component.

Telecommunication:

TAWASOL GROUP HOLDING: This Telecom stock was selected primarily for its extraordinary liquidity (**~28,899 avg. volume**), the highest in its sector, ensuring maximum flexibility. While its fundamental size is moderate (**~64.8 million TND**), its technical profile was highly active and promising. The stock showed a clear breakout from a long-term consolidation range with a massive increase in volume, indicating strong institutional or speculative interest and significant short-term momentum potential.

Sector Selection Based on Economic Analysis

The macroeconomic analysis guided sector selection by highlighting differences in resilience, growth drivers, and exposure to structural risks:

- **Defensive Sectors (Consumer Staples, Select Banks):**

High unemployment and moderate inflation favor sectors with stable and inelastic demand. Consumer staples and large, liquid banks were prioritized for their resilience and market stability.

- **Growth-Oriented Sectors (Export-Oriented Industrials, Agro-Industry):**

The recovery in agriculture and improving export performance, supported by a weak dinar, favor sectors with strong external demand and limited reliance on domestic investment.

- **Underweighted Sectors (Capital-Intensive and Import-Dependent Sectors):**

The sharp decline in the investment-to-GDP ratio and persistent external imbalances negatively affect construction-related and import-dependent sectors, which were therefore underweighted.

This allocation ensures alignment between macroeconomic conditions and sector exposure while remaining suitable for a short-term investment horizon.

Company Selection Using Financial Market Metrics

After identifying priority sectors based on the macroeconomic analysis and GDP trends discussed earlier, company selection was conducted using **financial market-based criteria derived from TSE data**. The objective was to construct a portfolio that is **liquid, implementable, and compatible with a short-term (3-month) investment horizon**.

1. Selection Criteria

a. Market Capitalization (Primary Criterion):

Companies with **larger market capitalizations** were prioritized, as they generally offer greater stability, stronger market presence, and lower idiosyncratic risk. This criterion was used as the **first filtering step** to eliminate small and illiquid firms, particularly in sectors affected by macroeconomic fragility.

b. Average Trading Volume (Liquidity Filter):

Following the market capitalization filter, stocks were further screened based on their **average daily trading volume**, ensuring sufficient liquidity for timely entry, exit, and potential rebalancing. This criterion was essential given the size and liquidity constraints of the Tunis Stock Exchange and the active nature of the strategy.

c. Fundamental analysis:

Each shortlisted company was assessed on key financial indicators such as profitability, earnings stability, leverage, and dividend history. Firms with solid and consistent financial performance were prioritized, while highly leveraged or unstable companies were excluded. This ensured that only fundamentally sound stocks advanced to the technical screening stage.

d. Technical Analysis & Chart Performance (Entry & Timing Criterion):

The final and decisive layer of selection was technical analysis. For each stock that passed the fundamental and liquidity screens, we examined price charts, trend patterns, volume behavior, and key indicators (e.g., moving averages, RSI, breakout patterns). Stocks were chosen based on compelling technical setups—such as breakouts from consolidation, bullish reversals, or strong momentum alignment—that indicated a higher probability of near-term price appreciation. This step ensured our entries were strategically timed to capture performance momentum.

2. Practical Portfolio Constraints

● Liquidity and Executability:

Preference was given to stocks combining high market capitalization with consistently strong trading volumes, such as **Attijari Bank, Poulina Group Holding, and One Tech Holding**, which facilitated efficient portfolio implementation.

● Budget Considerations:

The portfolio was constructed under a fixed capital constraint of **406.912 TND** (as reflected in TUNINVEST) reflecting realistic trading conditions for individual investors.

During execution, the total invested amount reached **373.62 TND**. This difference between available capital and invested funds is mainly explained by:

- Brokerage fees, even after the reduced commission agreement.

- Minor price movements between portfolio design and order execution.
- Rounding constraints, since fractional shares cannot be purchased.
- Markowitz optimization outputs, which determined target weights that could not be perfectly replicated in whole-share quantities.

To respect this constraint, stock selection focused on:

- **Reasonably priced shares**, enabling diversification,
- **High liquidity**, to minimize execution costs and slippage.
- **Controlled position sizing**, avoiding excessive exposure to any single stock.
- This slight overrun is operationally realistic and does not affect the portfolio's overall risk profile or investment strategy.

Sectoral Classification and Selected Companies Justification

Our sectoral classification and company selection were designed to ensure **diversification across the Tunis Stock Exchange (TSE)** while remaining fully aligned with the macroeconomic and sectoral analysis developed earlier in the report.

Rather than aiming for equal representation across all sectors, we **deliberately overweighted sectors exhibiting stronger resilience, recovery potential, or defensive characteristics**, such as consumer goods, export-oriented industries, banking, and healthcare. Sectors facing structural or cyclical weaknesses were included selectively, primarily for diversification or liquidity considerations.

Methodology and Data Sources

- Our stock selection followed a **top-down investment approach**. We first analyzed the macroeconomic and sector outlook, then narrowed the universe to **20 stocks** using three key filters: **fundamental strength, technical momentum, and liquidity**. Fundamental indicators were used to assess profitability and financial stability, while technical analysis helped identify favorable short-term price trends. A liquidity screen ensured that all selected stocks could be traded efficiently.
- The final portfolio was then structured in line with the principles of **Modern Portfolio Theory (Markowitz)**, with a strong emphasis on **sector diversification** to reduce unsystematic risk and improve risk-adjusted returns.
- We initially considered all companies listed on the **Tunis Stock Exchange**, then gradually narrowed the list by excluding stocks that did not meet our financial, liquidity, and budget criteria.

Ranking and Selection Process

- Stocks were first **ranked by market capitalization**, which served as the primary indicator of financial stability and market leadership.
- A second filter was applied based on **average trading volume**, ensuring sufficient liquidity and ease of execution within a short investment horizon.
- Additional qualitative screening incorporated **fundamental strength** and **technical signals**, allowing us to prioritize stocks combining stability, tradability, and momentum.
- This structured approach ensured that the final portfolio reflected both **macroeconomic signals** and **financial market realities**, resulting in a diversified yet selective exposure across sectors.

Tunis Consumer Goods		
Nom	market cap	average volume
POULINA GROUP HOLDING	2.65B	41.403K
SFBT	3.43B	35.057K
SAH	1.05B	25.815K
DELICE HOLDING	0.67B	12.102K
EURO-CYCLES	0.13B	7.444K
LAND'OR	0.14B	3.669K
NEW BODY LINE	0.02B	2.060K
STIP	0.01B	0.310K

Building Construct Materials		
Nom	market cap	average volume
CARTHAGE CEMENT	0.65B	108.90K
SOMOCER	0.02B	22.262K
MPBS	0.08B	8.969K
ESSOUKNA	0.01B	3.351K
SITS	0.06B	2.434K
CIMENTS DE BIZERTE	0.02B	1.152K
SANIMED	0.01B	0.418K
SIMPAR	0.04B	0.011K
SOTEMAIL	0.09B	0.008K

Health		
Nom	market cap	average volume
UNIMED	0.26B	36.948K
SIPHAT	0.01B	0.086K

BANKS		
Nom	market cap	average volume
BIAT	4.20B	2.794K
BANQUE ATTIJARI DE TUNIS	2.61B	5.807K
AMEN BANK	1.61B	13.244K
BANQUE DE TUNISIE	1.44B	24.910K
BANQUE NATIONALE AGRICOLE	0.70B	20.838K
UIB	0.70B	3.755K
STB BANK	0.60B	29.984K
BH BANK	0.45B	0.010K
ARAB TUNISIAN BANK	0.43B	35.529K
UBCI	0.38B	0.010K
WIFAK BANK INTERNATIONAL	0.24B	0.360K
BTE (ADP)	0.02B	0.478K

Distribution		
Nom	market cap	average volume
ENNAKL AUTOMOBILES	0.45B	1.264K
ARTES	0.44B	8.870K
CITY CARS	0.35B	2.355K
SMART TUNISIE	0.17B	4.861K
SOTRAPIL	0.09B	1.250K
STA	0.07B	3.604K
TUNISAIR	0.04B	8.506K
UADH	0.02B	7.736K

Tunis Industrials Components		
Nom	market cap	average volume
ASSAD	0.05B	31.712K
ONE TECH HOLDING	0.72B	26.172K
SOTUVER	0.55B	12.977K
SIAME	0.04B	8.572K
SAM	0.03B	4.413K
Financial Services		
Nom	market cap	average volume
ARAB TUNISIAN LEASE	0.20B	14.410K
HANNIBAL LEASE	0.08B	1.961K
BEST LEASE	0.07B	1.139K
BH LEASING	0.03B	1.097K
TUNINVEST - SICAR	0.01B	0.376K
ATTIJARI LEASING	0.02B	0.041K
CIL	0.19B	0.024K
TUNISIE LEASING & FACTORING	0.37B	0.022K
PLACEMENTS DE TUNISIE - SICAF	0.01B	0.019K
SPDIT - SICAF	0.35B	0.006K
Telecommunications		
Nom	market cap	average volume
TAWASOL GROUP HOLDING	0.06B	28.899K
TELNET HOLDING	0.08B	7.535K
SOTETEL	0.02B	7.529K
CELLCOM	0.01B	3.571K
AE TECH	0.00B	0.138K

Insurance		
Nom	market cap	average volume
BNA ASSURANCES	0.27B	15.544K
TUNIS RE	0.18B	1.266K
STAR	0.52B	0.316K
ASTREE	0.27B	0.065K
ASSURANCES MAGHREBIA	0.23B	0.034K
MAGHREBIA VIE	0.20B	0.027K
BH ASSURANCE	0.17B	0.014K
Tunis Basic Materials		
Nom	market cap	average volume
TPR	0.54B	29.572K
SOTIPAPIER	0.09B	8.112K
ICF	0.19B	1.380K
AIR LIQUIDE TUNISIE	0.25B	0.043K
ALKIMA	0.03B	0.022K
Tunis Consumer services		
Nom	market cap	average volume
SOTUMAG	0.11B	3.165K
MONOPRIX	0.16B	2.395K
MAGASIN GENERAL	0.13B	0.171K

Financials

Financial Services

Selected Companies:

1. **Hannibal Lease** (Market Cap: 0.08B TND, Avg. Volume: 1.961K)

Weight Justification:

Financial services received limited weight (**1 stock**) due to weak private investment and cautious credit conditions. Hannibal Lease was selected for its adequate liquidity and focused leasing activity, allowing controlled exposure to the sector while limiting risk.

Insurance

Selected Companies:

1. **Tunis Re** (Market Cap: 0.18B TND, Avg. Volume: 1.266K)
2. **Maghrebia Vie** (Market Cap: 0.20BTND, Avg. Volume: 0.027K)
3. **ASSURANCES Maghrebia** (Market Cap: 0.23B TND, Avg. Volume: 0.034K)

Weight Justification:

Insurance received good representation (**3 stocks**) due to its defensive characteristics and stable premium-based revenues. The selected companies provide diversification within the financial sector while limiting sensitivity to credit cycles and short-term economic volatility.

Banks

Selected Company:

1. **Banque Attijari De Tunisie** (Market Cap: 2.61B TND, Avg. Volume: 5.807K)
2. **BIAT** (Market Cap: 4.20B TND, Avg. Volume: 2.794K)
3. **AMEN BANK** (Market Cap: 1.61B TND, Avg. Volume: 13.244K)

Weight Justification:

The banking sector received good representation (**3 stocks**) due to its systemic importance, high market capitalization, and superior liquidity. These banks provide core portfolio stability and benefit from their dominant market positions, while their high trading volumes ensure efficient execution within a short investment horizon.

Industrials

Selected Companies:

1. **OneTech Holding** (Market Cap: 0.72B TND, Avg. Volume: 26.172K)

Weight Justification:

Industrials received selective representation (**1 stock**), reflecting the sector's uneven recovery and weak domestic investment. OneTech Holding was chosen for its strong fundamentals, export orientation, and high liquidity, providing targeted exposure while limiting cyclical risk.

Consumer Goods

Selected Companies:

1. **Poulina Group Holding** (Market Cap: 2.65B TND, Avg. Volume: 41.403K)
2. **Délice Holding** (Market Cap: 0.67B TND, Avg. Volume: 12.102K)
3. **SAH** (Market Cap: 1.05B TND, Avg. Volume: 25.815K)
4. **EURO-CYCLES** (Market Cap: 0.13B TND, Avg. Volume: 7.444K)

Weight Justification:

Consumer goods received strong representation (**4stocks**) due to their defensive nature, stable demand, and strong pricing power in an inflationary environment. The selected companies combine large market capitalization and high liquidity, providing portfolio stability while benefiting from resilient consumption and margin protection at the peak phase of the cycle.

Distribution

Selected Companies:

1. **City Cars** (Market Cap: 0.35B TND, Avg. Volume: 2.355K)
2. **SMART TUNISIE** (Market Cap: 0.17B TND, Avg. Volume: 4.861K)

Weight Justification:

The distribution sector received moderate weight

(**2 stocks**) due to its ongoing expansion and adequate liquidity, offering balanced exposure while limiting risk from weak domestic demand.

TELECOMMUNICATIONS

Selected Company:

1. **TAWASOL GROUP HOLDING**(Market Cap: 0.06B TND, Avg. Volume: 28.899K)
2. **SOTETEL**(Market Cap: 0.02 TND, Avg. Volume: 7.529K)

Weight Justification:

The telecommunications sector received moderate weight (**2 stocks**) due to its stable demand and strong liquidity, with TAWASOL Group Holding providing high tradability and SOTETEL offering diversified exposure within the sector.

HEALTH

Selected Company:

1. **UNIMED** (Market Cap: 0.26B TND, Avg. Volume: 36.948K)

Weight Justification:

The health sector received selective representation

(1 stock) due to its defensive nature, with UNIMED chosen for its strong fundamentals and high liquidity.

Building and Construction Materials

Selected Company:

1. **Carthage Cement** (Market Cap: 0.65B TND, Avg. Volume: 108.90K)
2. **ESSOUKNA** (Market Cap: 0.01B TND, Avg. Volume: 3.351K)
3. **CIMENTS DE BIZERTE** (Market Cap: 0.02B TND, Avg. Volume: 1.152K)

Weight Justification:

The sector received moderate weight **(3 stocks)** due to its gradual recovery, with Carthage Cement providing strong liquidity and ESSOUKNA and Ciments de Bizerte offering additional exposure to the construction cycle.

20 Stock Selection

The final 20-stock portfolio balances sectors with defensive, recovery, and expansion characteristics. **Banks and consumer goods** offer defensive stability, **distribution, telecommunications, and construction materials** target expansion, and **financial services, insurance, industrials, and health** provide selective exposure during recovery.

Step 4: Adjusted Return Calculations

This step is summarized in the excel sheet titled: “Stocks Daily Returns”.

This Excel worksheet presents a comprehensive evaluation of stock performance over a ten-year horizon, using both daily and annual return and volatility measures. For each stock, closing and adjusted closing prices are used to compute daily percentage returns. The data is then aggregated to derive key performance indicators, including average daily and annual returns, as well as daily and annual volatility measured by standard deviation. These indicators offer a clear assessment of each stock’s risk–return profile and support informed decisions regarding its inclusion in the portfolio.

Step 5: 10 Stocks Selection Process

Fundamental Analysis and Scoring Framework

To reduce the initial universe of 80 listed companies, we first applied our fundamental- and market-based scoring framework. This screening process allowed us to retain a shortlist of **20 stocks** that demonstrated solid financial strength, acceptable risk profiles, and sufficient liquidity. From this refined list, we then applied a **sector-diversification filter** to avoid excessive exposure to any single industry. Within each sector, the strongest candidates were selected based on their relative fundamentals and market outlook, resulting in a **final portfolio of 10 stocks** that combined stability, diversification, and attractive short-term trading potential—consistent with our moderate risk tolerance and three-month investment horizon. Each company was evaluated using a set of **quantitative indicators**, including profitability, balance-sheet quality, sector positioning, liquidity, and recent price momentum. These metrics were translated into a **custom scoring system**, allowing us to compare companies consistently across sectors and avoid subjective bias in the decision-making process.

The highest-scoring stocks were selected for inclusion in the final portfolio, ensuring that our investment choices were **data-driven, disciplined, and fully consistent with the macroeconomic and sectoral signals identified earlier in the report.**

1. Data Collection

Financial data for the 20 selected companies were obtained from the **BVMT (Bourse de Valeurs Mobilières de Tunis)** through the **Consolidated Annual Reports (Rapports Annuels Consolidés)** for the period **2020–2024**. From these reports, we extracted the key accounting figures required for our analysis, including :

- **Net Income, Shareholders' Equity, Total Liabilities, Revenues , and the Number of Outstanding Shares.**

2. Calculation of Key Metrics

After collecting the financial and market data for each company, we computed a set of core financial and performance indicators. These ratios were used to assess profitability, leverage, valuation, and short-term price momentum, and they form the analytical basis of our stock-selection scoring model.

2.1. Return on Equity (ROE)

ROE measures how efficiently a company generates profit from shareholders' equity.

Formula:

$$\text{ROE} = \text{Net Income} / \text{Shareholders' Equity}$$

2.2. Earnings Per Share (EPS)

EPS indicates the profit attributable to each outstanding share.

Formula:

$$\text{EPS} = \text{Net Income} / \text{Number of Outstanding Shares}$$

2.3. Net Profit Margin

Net Profit Margin evaluates how much profit is generated from total revenues.

Formula:

$$\text{Net Profit Margin} = \text{Net Income} / \text{Revenues}$$

2.4. EPS Growth

EPS Growth measures the change in earnings per share over time and captures earnings momentum.

Formula:

$$\text{EPS Growth} = \text{Change in EPS over the period}$$

2.5. Debt-to-Equity Ratio (D/E)

The D/E ratio assesses the company's financial leverage and capital structure risk.

Formula:

$$\text{Debt-to-Equity} = \text{Total Liabilities} / \text{Shareholders' Equity}$$

2.6. Price-to-Earnings Ratio (P/E)

The P/E ratio measures the company's valuation relative to its earnings.

Formula:

$$\text{P/E Ratio} = \text{Share Price} / \text{EPS}$$

2.7. RSI-14

RSI-14 is a technical momentum indicator calculated over 14 periods. It helps identify overbought and oversold price conditions.

Compute the price changes

$$\text{Change}_t = \text{Close}_t - \text{Close}_{t-1}$$

Separate gains and losses

$$\text{Gain}_t = \begin{cases} \text{Change}_t & \text{if } \text{Change}_t > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Loss}_t = \begin{cases} |\text{Change}_t| & \text{if } \text{Change}_t < 0 \\ 0 & \text{otherwise} \end{cases}$$

Compute the 14-period averages
(using Wilder's smoothing)

$$\sum_{i=1}^{14} \text{Gain}_i$$

2.8. On-Balance Volume (OBV)

OBV measures how volume flows with price movements.

Formula:

$$OBV_t = \begin{cases} OBV_{t-1} + Volume_t & \text{if } Close_t > Close_{t-1} \\ OBV_{t-1} - Volume_t & \text{if } Close_t < Close_{t-1} \\ OBV_{t-1} & \text{if } Close_t = Close_{t-1} \end{cases}$$

Usually:

$$OBV_0 = 0$$

2.9. Moving Average Convergence Divergence (MACD)

MACD is based on exponential moving averages

a) EMA formula (general)

$$EMA_t = \alpha \cdot Price_t + (1 - \alpha) \cdot EMA_{t-1}$$

where

$$\alpha = \frac{2}{n + 1}$$

b) MACD Line

2.10. Simple Moving Averages

General formula for an n -period SMA:

$$SMA_{n,t} = \frac{Close_t + Close_{t-1} + \cdots + Close_{t-n+1}}{n}$$

So for example:

- SMA-10

3. Data Normalization

To ensure that financial metrics measured on different scales could be compared objectively across companies, we applied **min–max normalization** to the following ratios: **ROE, EPS, Net Profit Margin, EPS Growth, and P/E**.

For each metric, the normalized score was calculated as:

$$\text{Normalized Value} = (\text{Value} - \text{Minimum}) / (\text{Maximum} - \text{Minimum})$$

where the minimum and maximum values correspond to the lowest and highest values of that metric across all companies in the sample.

To reduce the effect of short-term fluctuations, we computed the **average normalized value over the period 2020–2024** for each company. These normalized multi-year averages were then used as inputs to the scoring model.

4. Scoring RSI 14:

Unlike other financial ratios, the RSI-14 was not normalized using the min-max method. Instead, it was converted into discrete scores to reflect short-term market momentum:

1. **RSI < 30:** Assigned a score of **100**, indicating the stock is oversold and may present a buying opportunity.
2. **30 ≤ RSI ≤ 70:** Assigned a score of **50**, reflecting neutral momentum with no strong buy or sell signal.
3. **RSI > 70:** Assigned a score of **0**, indicating the stock is overbought and unfavorable for buying.

This categorization allowed the **RSI-14** to contribute to the scoring system in a way that emphasizes momentum signals, ensuring that highly oversold stocks received maximum consideration while overbought stocks were down weighted.

Momentum Score:

(35% RSI, 35% MACD, 15% SMA, 15% OBV) Emphasizes the speed and strength of recent price movements. A high score indicates strong, unified bullish momentum.

Trend Score:

(35% SMA, 25% MACD, 25% OBV, 15% RSI) Evaluates the persistence and quality of the price trend. A high score indicates a strong, well-confirmed uptrend.

Volume Score:

(45% OBV, 25% SMA, 15% MACD, 15% RSI) Measures the level of volume confirmation behind price action. A high score indicates substantial volume supporting the trend.

Equal-Weight Score:

Serves as a neutral baseline, calculated as the simple average of all four indicator scores.

4. Overall Composite Score (0-100)

The Composite Score is the arithmetic mean of the four category scores (Momentum, Trend, Volume, and Equal-Weight). This single value summarizes the entire technical picture.

≥ 70: Indicates strong bullish conditions across momentum, trend, and volume.

55 - 70: Indicates neutral or moderately bullish conditions.

≤ 55: Suggests weakening technical conditions or a bearish bias.

5. Weighting System

Stocks were classified as **liquid** or **illiquid**, and the scoring weights were adjusted accordingly. Liquid stocks received more weight on momentum and technical indicators, while illiquid

stocks were weighted more heavily on fundamental strength. This approach ensured the rankings reflect both tradability and investment potential, supporting risk-adjusted performance for our short-term horizon.

1. Illiquid Stocks:

- **Weights:**

Metric	Weight	Justification
Earnings Per Share (EPS)	0.10	EPS was given a relatively low weight because, on its own, it does not fully capture short-term market performance or future earnings dynamics.
EPS Growth	0.15	EPS Growth helps identify companies with improving profitability and positive earnings momentum, which is important for short-term performance.
Net Profit Margin	0.20	This metric was weighted more heavily as it reflects operational efficiency and the company's ability to sustain profitability, even during periods of inflationary pressure.
Return on Equity (ROE)	0.30	ROE received the highest weight because it captures overall financial performance and management effectiveness in generating returns for shareholders, aligning strongly with value creation.
Debt-To-Equity ratio	0.20	This ratio was strongly weighted to account for financial risk and leverage exposure, which is particularly important in Tunisia's high-interest-rate and low-investment environment.
Price-To-Equity ratio	0.05	P/E was given the lowest weight, as short-term performance is influenced more by earnings dynamics and financial strength than by relative valuation alone.

The scoring system incorporated these weights as follows:

$$\text{Final Score} = (\text{EPS} \times 0.10) + (\text{EPS Growth} \times 0.15) + (\text{Net Profit Margin} \times 0.20) + (\text{ROE} \times 0.30) + (\text{D/E} \times 0.20) + (\text{P/E} \times 0.05)$$

2. liquid Stocks:

- Weights:

Metric	Weight	Justification
Earnings Per Share (EPS)	0.10	Reflects profitability per share, moderately important for short-term trading.
EPS Growth	0.20	Highlights earnings improvement, strongly influencing market momentum.
Net Profit Margin	0.15	Shows efficiency, moderately affects price movements.
Return on Equity (ROE)	0.25	Measures return on equity, crucial for investor confidence and short-term trading.
Debt-To-Equity ratio	0.15	Indicates financial leverage, moderately impacts risk perception.
Price-To-Equity ratio	0.15	Captures valuation, guiding short-term buy/sell decisions. relative valuation alone.

The scoring system incorporated these weights as follows:

$$\text{Final Score} = (\text{EPS} \times 0.10) + (\text{EPS Growth} \times 0.20) + (\text{Net Profit Margin} \times 0.15) + (\text{ROE} \times 0.25) + (\text{D/E} \times 0.15) + (\text{P/E} \times 0.15)$$

6. Ranking and Selection

- The stock selection process followed a multi-stage filtering approach. First, an initial universe of **approximately 80 stocks** was screened using **fundamental criteria**, to exclude financially weak or high-risk firms. Only companies demonstrating acceptable profitability, stability, and financial soundness were retained.
- From this group, a **shortlist of 20 stocks** was then identified based on **technical analysis and market behavior** (trend strength, momentum indicators, chart structure,

and liquidity). This ensured that the selected companies were not only fundamentally solid but also showed favorable price dynamics for our investment horizon.

- Finally, the **top 10 portfolio constituents** were chosen with a strong emphasis on **sector diversification**. We reviewed the main sectors represented on the Tunis Stock Exchange and selected the most suitable stock within each sector at the time, in order to avoid concentration risk and improve portfolio stability.
- These 10 stocks were then ranked according to their **final composite scores from the weighting model**, which determined their relative attractiveness and portfolio allocation.
- This ranking system allowed us to focus on the companies demonstrating the strongest financial performance and the most favorable short-term momentum, consistent with our investment objectives.

All calculations and rankings are documented in the Excel workbook titled “**Fundamental Analysis and Scoring System**”, which is attached to this report.

Step 6: CAPM and Portfolio Optimization

To optimize the portfolio, we determined the optimal weight of each stock by maximizing the target Sharpe Ratio, in line with a moderate risk profile.

1. Data Collection and Alignment

- Daily adjusted return data were obtained from **Tunisie Valeurs**. Because some newly listed stocks such as **AMV and SMART Tunisie** only have reliable trading data starting in **2023**, the dataset was aligned to the **earliest common period available across all securities**. Therefore, daily returns were collected for all selected stocks together with **TUNINDEX as the market benchmark** from **January 2023 to 30 September 2025**.
- Aligning the series to a common start date was essential because **Markowitz portfolio optimization and CAPM require returns measured over identical time periods** in order to correctly estimate **variances, covariances, and correlations**. Using this consistent dataset, we constructed the correlation matrix and variance–covariance matrix, which were then used to compute the optimal portfolio weights under Modern Portfolio Theory.
- This approach allowed us to **apply Markowitz principles while accounting for the limited historical data of newly listed firms**, ensuring methodological rigor and comparability across all assets.

2. CAPM Calculations

Based on the aligned dataset, key risk and return indicators were computed for each selected stock within the CAPM framework, including:

- **Beta**
- **Covariance with the market**
- **Daily and annual standard deviation (volatility)**
- **Daily and annual mean returns**
- **Sharpe Ratio**

These metrics allowed us to evaluate each stock's contribution to portfolio risk and return and to construct an optimized allocation.

3. Markowitz Optimal Portfolio

1. Covariance Matrix

We computed the covariance matrix of the 10 selected stocks to capture the interactions and co-movements between their returns. This matrix is essential for evaluating overall portfolio risk and diversification benefits.

2. Optimization Objective

The main objective was to maximize the portfolio's Sharpe Ratio, ensuring consistency with our moderate risk tolerance. This approach allows:

- **Maximization of risk-adjusted returns.**
- **Prevention of excessive concentration in high-risk stocks by optimizing the portfolio as a whole rather than individual securities.**

3. Determination of Optimal Weights

Using the estimated mean returns and covariance matrix, optimal portfolio weights were obtained through **Excel's Solver tool**, which identifies the allocation that maximizes the portfolio's risk-adjusted performance.

Companies	Tawasol GH	Smart Tunisie	Attijari Bank	Euro-cycles	UNIMED	AMV	Carthage Cement	Poulina	One Tech Holding	Essoukna	Total
Nber of stock to purchase	0	1	2	0	1	6	0	5	9	1	
Stock's price	0.83	18.2	66.9	12.9		6.75	1.94	18.1	8.88	3.06	
weights	0	0.0131998	0.488489	0	0.00731	0.08587	0	0.21113	0.191427295	0.0025738	
Money invested	0	18.2	133.8	0	0	40.5	0	90.5	79.92	3.06	365.98

4. Portfolio Target Mean Return and Risk

Using the optimized portfolio weights, we computed the portfolio's target expected return and target risk, measured by the standard deviation of returns. These indicators summarize the portfolio's expected performance and volatility under the optimal allocation.

Portfolio mean yearly return	0.30434019
portfolio yearly std	0.12206428

5. Deliverables

The entire analysis is compiled in an Excel file titled “ **CAPM and Markowitz**”, which includes:

- Daily return data for all selected stocks.
- Calculations of CAPM metrics, including Beta and Sharpe Ratio.
- Covariance matrix and Markowitz portfolio optimization results.
- Portfolio expected return and risk (**standard deviation**).

Step 7: Market Entry Strategy and Order Execution

We implemented a comprehensive approach to determine optimal market entry points for each stock in our portfolio, consistent with our short-term investment horizon and using technical analysis.

1. Technical Analysis Indicators:

- **MACD 12-26:** Tracks trend changes and generates potential buy signals when the MACD line crosses above the signal line.
- **RSI-14:** Identifies momentum conditions and confirms MACD signals. **RSI < 30** indicates oversold conditions (potential buy), while **RSI > 70** signals overbought conditions (avoid buying or consider selling).
- **OBV (On-Balance Volume):** Used to measure buying and selling pressure by combining price and trading volume. For each stock, OBV was analyzed to determine:

- **Trend Signal:** Comparing recent OBV values with previous levels, we identified whether the stock exhibited a bullish, bearish, or neutral trend.
- **Trend Strength:** The magnitude of OBV change was used to classify trends as strong, weak, or neutral, helping to assess the intensity of market momentum.
- **Price–OBV Alignment:** OBV was compared with price movements to confirm trends. When OBV and price moved in the same direction, the trend was confirmed (bullish or bearish). Divergences between price and OBV signaled potential reversals (bullish or bearish divergence).

Daily price data for the past 10 years up to the project’s last trading day were prepared in Excel. A custom Python program was developed to generate charts for MACD, RSI-14, OBV, and price data, as well as relevant buy and sell signals.

2. Trading Volume:

Daily trading volumes were monitored to validate buy signals. High volumes confirmed market activity and investor interest, while low-volume stocks were avoided to reduce illiquidity risk and price volatility.

3. Cash Reserve Strategy

As part of our risk management, a portion of the portfolio was held in cash to support stability and flexibility. This approach allowed us to:

- Limit exposure during sudden market swings.
- Ensure funds were available for timely trades.
- Reduce reliance on illiquid securities.
- Cover unexpected expenses or transaction costs.

3. Entry Point Determination

While the technical dataset used in the analysis was available only up to **30 September 2025**, we complemented it with price charts and adjusted-close data from our broker’s trading platform. This allowed us to continue monitoring market behavior in real time. Entry points were then determined using technical indicators, such as moving averages, RSI, and price-trend signals, to identify favorable buying moments aligned with momentum and trend confirmation.

1. Order Book Analysis

- We carefully reviewed the **order book** to observe the range of prices currently being bid and offered by market participants.
- This analysis helped us understand **market depth, liquidity, and short-term price dynamics**, allowing us to avoid entering at an unfavorable price.

2. Use of Limit Orders

- To control our entry price, we **exclusively used limit orders** rather than market orders.
- During the trading period, we observed **significant price volatility and frequent price jumps**, which increased the risk of overpaying if using market orders.
- By setting a limit order, we ensured that:
 - We **did not pay more than our predefined acceptable price**.
 - Our entries remained **aligned with our risk-management approach and conservative trading style**.

Limit Order Execution for Higher Allocation Stocks

For positions where the optimal portfolio allocation required purchasing more than one share, limit orders were used to ensure price discipline and proper execution.

- **Attijari Bank:**
 - The target allocation required the purchase of **2 shares**, which were submitted as a limit order:
 - **2 shares at 133.80 TND**, executed successfully due to sufficient market liquidity.
- **Maghrebiana Vie:**
 - To reach the desired portfolio weight, **6 shares** were purchased:
 - **6 shares at 40.50 TND**, fully executed at the specified price.
- **One Tech Holding:**
 - A total of **9 shares** was needed to satisfy the allocation requirement:
 - **9 shares at 79.20 TND**, executed without difficulty, reflecting adequate trading activity.
- **Poulina Group Holding:**
 - The optimized allocation implied the purchase of **5 shares**:
 - **5 shares at 89.45 TND**, executed at the limit price.

15/12/2025 Acquisition	ATTIJARI BANK	(Quantité : 2,Prix : 66.900)	134.358	
15/12/2025 Timbre fiscal			1.000	
15/12/2025 Rachat	TUNISIE SICAV	(Quantité : 2,Prix : 131.024)		262.048
15/12/2025 Acquisition	ONE TECH HOLDING	(Quantité : 9,Prix : 8.800)	79.639	
15/12/2025 Acquisition	MAGHREBIA VIE	(Quantité : 6,Prix : 6.750)	40.870	
15/12/2025 Acquisition	UNIMED	(Quantité : 1,Prix : 9.020)	9.334	
15/12/2025 Acquisition	POULINA G H	(Quantité : 5,Prix : 17.890)	89.907	
15/12/2025 Rachat	TUNISIE SICAV	(Quantité : 1,Prix : 131.024)		131.024
15/12/2025 Acquisition	SMART TUNISIE	(Quantité : 1,Prix : 18.580)	18.911	
15/12/2025 Acquisition	ESSOUKNA	(Quantité : 1,Prix : 3.070)	3.374	
29/12/2025 Cession	UNIMED	(Quantité : 1,Prix : 9.140)		8.826
29/12/2025 Timbre fiscal			1.000	
29/12/2025 Cession	ONE TECH HOLDING	(Quantité : 9,Prix : 8.740)		78.222
29/12/2025 Cession	POULINA G H	(Quantité : 5,Prix : 18.450)		91.491
29/12/2025 Cession	MAGHREBIA VIE	(Quantité : 6,Prix : 6.800)		40.430
29/12/2025 Cession	ATTIJARI BANK	(Quantité : 2,Prix : 66.790)		133.024
29/12/2025 Cession	SMART TUNISIE	(Quantité : 1,Prix : 18.930)		18.599
29/12/2025 Cession	ESSOUKNA	(Quantité : 1,Prix : 3.140)		2.837

This execution approach allowed us to respect the optimized portfolio weights while maintaining control over transaction prices and adapting to market liquidity conditions.

3. Historical Price Analysis

We examined each stock's recent trading range, focusing on the highest and lowest prices over the past month, to assess short-term trends and select reasonable entry points. The final purchase prices are documented in the Excel file **"Portfolio Return Track."**

4. Intraday Monitoring

We tracked intraday price and volume fluctuations to improve our entry timing and avoid purchasing at peak prices.

Key Elements of Our Approach:

- Momentum-Oriented Trading:** Used MACD and RSI-14 to identify short-term trends.
- Controlled Risk:** Employed limit orders, checked trading volume, and kept cash on hand to limit exposure and reduce the risk of buying illiquid stocks.
- Informed Decisions:** Integrated technical signals, volume patterns, and order book data to guide purchase decisions.
- Adaptive Strategy:** Split limit orders and adjusted trades in real time to respond to changing market conditions.

5. Budget Allocation

At the beginning of the project, we deposited **406.912 TND** into our trading account. This amount represented the total funds available for both investment and trading-related costs.

1. Investment Budget

During the trading period, we built a portfolio whose **Ending Portfolio Value reached 376.50 TND.**

The Net Ending Portfolio recorded in our tracking sheet is **375.00 TND**. However, this figure does not account for broker transaction fees, which are charged at a rate of **0.2%** per transaction. Therefore, the true net value after fees would be slightly lower than the reported amount.

This amount represents the **capital that was actually invested in stocks during the project.**

2. Fees and Risk-Management Cash Reserve

From the initial 406.912 TND:

- Total transaction charges recorded during the project: **8.784 TND**
- 0.2% broker's fees column total: **1.50 TND**
- Cash still available at the end of the period:

401.948 TND

A portion of the capital therefore remained in cash throughout the project. This **cash reserve acted as a risk-management buffer**, ensuring we could cover all transaction costs and still keep liquidity available for future trades.

3. Final Performance

The cumulative portfolio performance shown in our return column is:

$(1 + \text{return}) = 100.77\%$

This means our portfolio generated a **total return of 0.77% over the period.**

Date	Beginning Portfolio Value	Ending Portfolio Value	Available to Invest	Total broker fees	Total Brokers fees(0.2%)	Daily unrealized capital loss/gain	return	(1+return)												
	0	0	406.912	0		-	-	-												
12/15/2025	373.62	372.68	29.52	4.712	0.75	-0.94	-0.25%	99.75%												
12/16/2025	372.68	372.28	29.52	4.712	0.75	-0.4	-0.11%	99.89%												
12/17/2025	372.28	372.28	29.52	4.712	0.75	0	0.00%	100.00%												
12/18/2025	372.28	373.95	29.52	4.712	0.75	1.67	0.45%	100.45%												
12/19/2025	373.95	373.7	29.52	4.712	0.75	-0.25	-0.07%	99.93%												
12/22/2025	373.7	373.06	29.52	4.712	0.75	-0.64	-0.17%	99.83%												
12/23/2025	373.06	372.52	29.52	4.712	0.75	-0.54	-0.14%	99.86%												
12/24/2025	372.52	371.98	29.52	4.712	0.75	-0.54	-0.14%	99.86%												
12/25/2025	371.98	372.93	29.52	4.712	0.75	0.95	0.26%	100.26%												
12/26/2025	372.93	374.48	29.52	4.712	0.75	1.55	0.42%	100.42%												
12/29/2025	374.48	376.5	401.948	8.784	1.50	2.02	0.54%	100.54%	100.77%											
Net Ending Portfolio		375.00																		
rf yearly	7.51%																			
rf daily	0.0198%																			
Average Daily Return	0.023%																			
Annualized Average Daily Return	8.50%																			
Daily Volatility	0.25%																			
Annualized Volatility	4.05%																			
TWR	0.023%																			
MWR	0.77%																			
Annualized MWR	20.54%																			
Sharpe ratio	0.24																			
Average unrealized capital gain/lc	0.086																			
diversification ratio	25%																			
12/15/2025	-373.62																			
12/29/2025	376.5																			

Note: For consistency, to annualize returns we used number of days equal to 365 instead of 252 and considered the number of days 15 (difference between starting day and ending day), since we are working with the risk free rate of zero-coupon bonds provided by Tunisie Clearing that was calculated using a base of 365 days.(source: <https://www.tunisieclearing.com/upload/202103/note%20corporate%20vf.pdf>)

Step 8: Portfolio Rebalancing

1. Managing Exposure to Attijari Bank:

Attijari Bank was selected for its high liquidity and ease of trading, making it a key holding in our portfolio. Over time, the stock's price gradually declined and that's why we decided to hold only 2 shares of this stock. To avoid excessive concentration risk and protect portfolio value, we chose to maintain our existing position without increasing or averaging down.

2. Monitoring UNIMED Due to Company-Specific Risk

We purchased 1 share of UNIMED at 9.02 TND. During the period, negative news concerning the company's leadership affected investor sentiment. To limit risk related to uncertainty, we chose not to add further exposure and kept the position unchanged.

3. Maintaining One Tech Holding Despite Losses

One Tech Holding experienced a decline in value during the trading period. Despite this short-term loss, we chose to retain the stock because we had confidence in its recovery potential and long-term upside. This reflects our strategy of holding fundamentally and technically promising assets even during temporary setbacks.

Management of Uninvested Cash

During the final week of December, increased market volatility reduced the availability of suitable investment opportunities. To avoid taking excessive risk, we chose not to deploy the remaining cash. Despite actively monitoring prices, volumes, and technical indicators (RSI and MACD), no opportunities met our liquidity and risk criteria, justifying the decision to maintain uninvested cash.

Justification of Asset Liquidation Decisions

The liquidation of Essoukna and Smart Tunisie shares was a strategic portfolio management decision despite unfavorable execution prices. Essoukna was sold at a return of -4.27% , which was anticipated due to its fixing mechanism and limited short-term upside. Given its very small portfolio weight (approximately 0.8%), the negative impact on overall portfolio performance was negligible, and retaining the stock would not have enabled the portfolio to outperform the market. Therefore, liquidation was justified.

Similarly, Smart Tunisie was liquidated at a marginal loss of approximately -0.1% , primarily due to a wide bid-ask spread observed on the trading day. Although the previous closing price was slightly higher, the stock's relatively small portfolio weight (around 5%) limited its impact on overall returns. Moreover, stronger performances from the remaining portfolio holdings offset this minor loss. Consequently, liquidating Smart Tunisie was consistent with the objective of fully unwinding the portfolio while preserving overall performance.

Step 9: Performance Analysis and Return Calculations

All metrics and calculations mentioned in this step are included in the excel file titled **“Portfolio Return Track”**

Adjustments in Portfolio Calculations

Portfolio adjustments were calculated by updating the portfolio value according to observed price changes and corresponding daily returns. Starting from an initial value of **372.62**, the adjusted portfolio value evolved through return compounding $(1+rt)$, reaching **376.5** by **29/12/2025**. Performance was measured using the **time-weighted return (TWR)** to isolate the effect of market movements, resulting in an **average daily return of 0.023%** and an **annualized return of 8.50%** . Risk was assessed through **annualized volatility of 4.05%** and a **Sharpe ratio of 0.25** , indicating positive but moderate risk-adjusted performance. The **25%**

diversification ratio confirms that the adjustment process effectively reduced unsystematic risk.

Return Calculations

To ensure reliable performance measurement, all portfolio metrics including returns, asset weights, and valuation were computed using **consistent and accurately adjusted data**, allowing the results to reflect the true behavior of the portfolio over time.

1. Individual Stock Return Calculations

a. Dynamic Portfolio Weight Determination

Portfolio weights were **updated daily** based on each asset's market value relative to the total portfolio value. This dynamic approach captures changes in prices and ensures that portfolio composition evolves with market conditions. The weight of asset i at time t was calculated as:

$$\text{Weight}_i = \frac{\text{Number of shares}_i \times \text{Closing price}_i}{\text{Total portfolio value}}$$

Application:

- Determination of portfolio composition (KRI)

b. Daily Stock Return Computation

Daily returns for each stock were calculated using **successive closing prices**, measuring relative price changes from one trading day to the next:

$$R_{i,t} = \frac{\text{Closing price}_t - \text{Closing price}_{t-1}}{\text{Closing price}_{t-1}}$$

These individual stock returns served as fundamental inputs for several performance and risk indicators.

Application:

- Average daily return of each stock (KPI)

- Daily return volatility (standard deviation)
- Diversification ratio (KRI)

2. Portfolio Returns Based on Total Portfolio Value

a. Daily Portfolio Valuation

The total portfolio value was recalculated **at the end of each trading day** by aggregating the market value of all held securities and adding any remaining cash balance. This approach ensures that portfolio valuation reflects current market prices and available liquidity. The daily portfolio value was computed as:

$$\text{Portfolio Value}_t = \sum_{i=1}^n (\text{Number of shares}_i \times \text{Closing price}_{i,t}) + \text{Available cash}$$

b. Portfolio Daily Return Measurement

Daily portfolio returns were derived from **period-to-period changes** in total portfolio value, capturing the combined impact of price movements across all assets:

$$R_{p,t} = \frac{\text{Portfolio Value}_t - \text{Portfolio Value}_{t-1}}{\text{Portfolio Value}_{t-1}}$$

These daily portfolio returns formed the basis for evaluating overall performance and risk.

Performance Indicators Derived

The calculated portfolio returns were used to compute the following key performance indicators (KPIs):

- **Money-Weighted Return (MWR):** Incorporates the timing and size of cash flows, reflecting the investor's actual portfolio experience
- **Annualized MWR**
- **Time-Weighted Return (TWR):** Eliminates the impact of cash flows to measure pure investment performance

- **Annualized TWR**
- **Maximum Drawdown (MDD):** Assesses downside risk
- **Portfolio Sharpe Ratio:** Evaluates risk-adjusted performance
- **Mean portfolio return**
- **Annualized mean portfolio return**

Benchmark Selection: TUNINDEX

The **TUNINDEX** was selected as the benchmark for evaluating portfolio performance due to its strong alignment with our investment strategy. As a broad market index composed of actively traded stocks listed on the **Tunisian Stock Exchange**, it provides a representative measure of overall market performance. Moreover, its focus on liquid securities and sensitivity to domestic economic conditions make it a relevant and reliable reference for comparing the portfolio's returns and risk characteristics.

Individual KPIs and KRIs

- The **Sharpe ratio**, **beta**, and **annual standard deviation** of each stock were estimated within the **CAPM framework** from **January 2023 until 30 septembre 2025** . In addition, the **daily standard deviation** observed during the trading period was calculated separately and is reported in the *“Portfolio Return Track”* Excel file.
- The **mean return of each stock** and its **portfolio weight** were derived directly from **actual trading activity data**, reflecting the realized performance and allocation decisions made during the investment period.

Portfolio Evaluation:

The data presented reflect real-time values and are based on our executed trading activities.

Behavioral Biases

1. Overconfidence:

- We were overly confident that **Attijari Bank's** stock price would rise, expecting quick gains. This expectation was based on our assumptions rather than thorough market analysis. Contrary to our belief, the stock declined, resulting in unexpected losses. Our overconfidence led us to act prematurely without waiting for clear market signals.

2. Loss Aversion:

- We held onto declining stocks such as **One Tech Holding and Unimed**, hoping they would recover. This decision was driven by an emotional reluctance to realize losses rather than objective market evidence. Ultimately, this approach exposed us to further declines, demonstrating how loss aversion affected our trading choices.

3. Anchoring Bias:

- Despite **Attijari Bank's** stock falling below levels, we were comfortable with, we anchored our expectations on its previous performance, believing it would rebound. Our decisions were influenced by past price trends rather than solid market analysis, leading to suboptimal trading outcomes.

Ending Notes

This project presented several challenges, highlighting that the transition from theoretical knowledge to practical application is rarely seamless. We faced obstacles such as setting limit prices too hastily and holding onto underperforming investments longer than advisable.

Despite these difficulties, we developed a range of practical skills, including data analysis, technical trading, and problem-solving. Interactions with our broker, for example, emphasized the importance of clear communication and professional collaboration.

Above all, this project offered an invaluable learning experience. It provided an opportunity to test our limits, refine our skills, and gain insights that will be highly beneficial for our future professional endeavors. We are grateful for the chance to participate in such a meaningful and practical exercise.

Appendix Python Technical Analysis Code and Data Sources

```
"""
Portfolio Performance Dashboard

This Streamlit app visualizes portfolio performance metrics, geometric returns,
stock trends, and portfolio composition using data from Excel.

# 📌 How to Install Required Libraries:
Run the following command in your terminal or command prompt:
    pip install streamlit pandas numpy plotly

# 📌 How to Run the Dashboard:
1. Open your terminal and navigate to the folder where the file is saved.
2. Run the Streamlit app using:
    streamlit run portfolio_dashboard.py
3. The dashboard will open in your default web browser.

# 📌 Requirements:
- Python 3.8 or higher
- Internet browser for viewing the dashboard
"""

import streamlit as st
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import warnings
warnings.filterwarnings('ignore')

st.set_page_config(
    page_title="Portfolio Return Dashboard",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.markdown("""
<style>
    .main-header {

```

```

        font-size: 2.5rem;
        color: #1E3A8A;
        text-align: center;
        margin-bottom: 1rem;
    }

    .sub-header {
        font-size: 1.5rem;
        color: #3B82F6;
        margin-bottom: 1rem;
    }

    .metric-card {
        background-color: #F8FAFC;
        padding: 1.5rem;
        border-radius: 10px;
        border-left: 5px solid #3B82F6;
        margin-bottom: 1rem;
        text-align: center;
    }

    .metric-card h3 {
        font-size: 1rem;
        color: #64748B;
        margin-bottom: 0.5rem;
    }

    .metric-card h2 {
        font-size: 2rem;
        color: #1E40AF;
        margin: 0;
    }

    .comparison-positive {
        color: #10B981;
        font-weight: bold;
    }

    .comparison-negative {
        color: #EF4444;
        font-weight: bold;
    }

    .data-table {
        background-color: white;
        border-radius: 10px;
        padding: 1rem;
        box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }
</style>
""" , unsafe_allow_html=True)

st.markdown("<h1 class='main-header'>📊 Portfolio Performance Dashboard</h1>", unsafe_allow_html=True)

```

```

with st.sidebar:
    st.markdown("## 🔍 Filters & Controls")

    st.markdown("### Stock Selection")
    all_stocks = ["One Tech Holding", "POULINA", "UNIMED", "AMV", "Attijari Bank", "ESSOUKNA", "SMART TUNISIE"]
    selected_stocks = st.multiselect(
        "Select stocks to display:",
        options=all_stocks,
        default=all_stocks[:3]
    )

    st.markdown("### Pie Chart Controls")
    show_pie_chart = st.checkbox("Show Portfolio Composition Pie Chart", value=True)

    st.markdown("### Display Options")
    show_daily_geometric = st.checkbox("Show Daily Geometric Returns Table", value=True)
    show_stock_performance = st.checkbox("Show Stock Performance", value=True)

@st.cache_data
def create_data():
    """Create data with exact values from Excel files."""

    dates = pd.to_datetime([
        '2025-12-15', '2025-12-16', '2025-12-17', '2025-12-18', '2025-12-19',
        '2025-12-22', '2025-12-23', '2025-12-24', '2025-12-25', '2025-12-26',
        '2025-12-29'
    ])

    portfolio_data = pd.DataFrame({
        'Date': dates,
        'Beginning_Value': [373.62, 372.68, 372.28, 372.28, 373.95, 373.7, 373.06, 372.52, 371.98, 372.93, 374.48],
        'Ending_Value': [372.68, 372.28, 372.28, 373.95, 373.7, 373.06, 372.52, 371.98, 372.93, 374.48, 376.50],
    })

    portfolio_data['Daily_Return'] = (portfolio_data['Ending_Value'] / portfolio_data['Beginning_Value']) - 1

    daily_returns_excel = [-0.0025, -0.0011, 0.0000, 0.0045, -0.0007, -0.0017, -0.0014, -0.0014, 0.0025, 0.0041,
0.0054]

    geometric_portfolio = [0.9975, 0.9964, 0.9964, 1.0009, 1.0002, 0.9985, 0.9971, 0.9957, 0.9982, 1.0023, 1.0076]
    geometric_tunindex = [0.9973, 0.9965, 0.9957, 0.9987, 1.0014, 1.0027, 1.0016, 1.0042, 1.0013, 1.0017, 1.0027]

```

```

tunindex_returns = [-0.0027, -0.0008, -0.0008, 0.0030, 0.0027, 0.0013, -0.0011, 0.0026, -0.0029, 0.0004, 0.0010]

geometric_data = pd.DataFrame({
    'Date': dates,
    'Portfolio_Geometric': geometric_portfolio,
    'TUNINDEX_Geometric': geometric_tunindex,
    'TUNINDEX_Return': tunindex_returns
})

stock_data = {}
stock_prices = {
    "One Tech Holding": [79.65, 79.2, 79.2, 80.1, 79.83, 79.65, 79.38, 78.75, 78.66, 77.85, 78.66],
    "POULINA": [90.0, 89.9, 89.9, 89.75, 90.0, 90.0, 90.45, 90.0, 91.25, 91.95, 92.25],
    "UNIMED": [9.0, 8.9, 8.9, 8.98, 8.9, 9.06, 9.0, 8.9, 8.9, 8.89, 9.14],
    "AMV": [40.38, 39.9, 39.9, 40.2, 40.8, 40.8, 40.2, 40.5, 40.26, 40.26, 40.8],
    "Attijari Bank": [132.0, 132.98, 132.98, 132.78, 131.98, 131.4, 131.4, 131.78, 131.78, 133.3, 133.58],
    "ESSOUKNA": [3.07, 3.09, 3.09, 3.15, 3.29, 3.15, 3.09, 3.1, 3.18, 3.28, 3.14],
    "SMART TUNISIE": [18.58, 18.31, 18.31, 18.99, 18.9, 19.0, 19.0, 18.95, 18.9, 18.95, 18.93]
}

for stock, prices in stock_prices.items():
    stock_df = pd.DataFrame({
        'Date': dates,
        'Close_Price': prices
    })
    stock_df['Daily_Return'] = stock_df['Close_Price'].pct_change().fillna(0)
    stock_data[stock] = stock_df

portfolio_weights = {}

weights_summary = {
    "One Tech Holding": [21.25147579693034, 21.39518641882454, 21.27430965939616, 21.17930204572804,
21.43430559272143,
21.39870262156222, 21.38140234081392, 21.33985698155815, 21.11656343013434,
21.00512710959196, 20.67729083665338],
    "POULINA": [24.00182462165933, 24.17535188567745, 24.14849038358225, 24.04064714534029, 24.01659084827402,
24.12480566128773, 24.15977665628691, 24.31582343136728, 24.13321534872496, 24.36712240974151,
24.42231075697211],
    "UNIMED": [2.420306965761511, 2.417535188567745, 2.390673686472548, 2.379997325845701, 2.402997056462403,
2.38567522650512, 2.432084183399549, 2.419484918544008, 2.386506851151691, 2.376628925443281,
2.361221779548472],
    "AMV": [10.86723194161211, 10.84667454604062, 10.71773933598367, 10.66987565182511, 10.75729194541076,
10.93657856645044, 10.9524320841834, 10.80703263616323, 10.85994690692623, 10.75090792565691,
10.69322709163346],

```

```

        "Attijari Bank": [35.90211441451111, 35.45718276566026, 35.72042548619319, 35.56090386415296,
35.53117473909553,
                        35.37768723529727, 35.27327391817889, 35.32447981074252, 35.33639020727751,
35.19013031403546, 35.40504648074369],
        "ESSOUKNA": [0.8237630138456584, 0.8246481143225528, 0.8300204147415924, 0.8263136782992379,
0.8429221300508429,
                        0.8818956736181848, 0.8455921829700419, 0.8306898220334427, 0.8312551953449709,
0.8491775261696218, 0.8711819389110224],
        "SMART TUNISIE": [4.985510357411183, 4.990867089287633, 4.9183410336306, 4.896376520925258,
5.081616269735082,
                        5.066209188870423, 5.100397294105015, 5.107801494704016, 5.081382565092644,
5.046998504593035, 5.033200531208498]
    }

    for stock, weight_values in weights_summary.items():
        portfolio_weights[stock] = pd.DataFrame({
            'Date': dates,
            'Weight': [w / 100 for w in weight_values] # Convert percentage to decimal
        })

    return portfolio_data, geometric_data, stock_data, portfolio_weights

portfolio_data, geometric_data, stock_data, portfolio_weights = create_data()

st.markdown("## 📊 Portfolio Performance Metrics")

portfolio_geometric_final = geometric_data['Portfolio_Geometric'].iloc[-1]
tunindex_geometric_final = geometric_data['TUNINDEX_Geometric'].iloc[-1]

portfolio_geometric_return = (portfolio_geometric_final - 1) * 100
tunindex_geometric_return = (tunindex_geometric_final - 1) * 100

outperformance = portfolio_geometric_return - tunindex_geometric_return

initial_value = portfolio_data['Beginning_Value'].iloc[0]
final_value = portfolio_data['Ending_Value'].iloc[-1]
cumulative_return_simple = ((final_value - initial_value) / initial_value) * 100
avg_daily_return = portfolio_data['Daily_Return'].mean() * 100
daily_volatility = portfolio_data['Daily_Return'].std() * 100

cumulative_returns = (1 + portfolio_data['Daily_Return']).cumprod()
running_max = cumulative_returns.expanding().max()
drawdown = (cumulative_returns - running_max) / running_max

```

```

max_drawdown = drawdown.min() * 100

st.markdown("### Geometric Returns Comparison (From Excel Columns J & P)")

col1, col2, col3, col4 = st.columns(4)

with col1:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Portfolio Geometric Return</h3>
        <h2>{portfolio_geometric_return:.4f}%</h2>
        <small>From Excel Column J</small>
    </div>
    """, unsafe_allow_html=True)

with col2:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>TUNINDEX Geometric Return</h3>
        <h2>{tunindex_geometric_return:.4f}%</h2>
        <small>From Excel Column P</small>
    </div>
    """, unsafe_allow_html=True)

with col3:
    outperformance_class = "comparison-positive" if outperformance > 0 else "comparison-negative"
    outperformance_text = f"+{outperformance:.4f}%" if outperformance > 0 else f"{outperformance:.4f}%"
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Outperformance</h3>
        <h2 class="{outperformance_class}">{outperformance_text}</h2>
        <small>Portfolio vs Benchmark</small>
    </div>
    """, unsafe_allow_html=True)

with col4:
    if outperformance > 0:
        result_text = "✓ Outperformed"
        result_color = "#10B981"
    else:
        result_text = "✗ Underperformed"
        result_color = "#EF4444"

    st.markdown(f"""
    <div class='metric-card'>
        <h3>Result</h3>
        <h2 style="color: {result_color}">{result_text}</h2>
        <small>vs TUNINDEX Benchmark</small>
    """, unsafe_allow_html=True)

```

```

    </div>
    """ , unsafe_allow_html=True)

st.markdown("### Other Performance Metrics")

col5, col6, col7, col8 = st.columns(4)

with col5:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Simple Cumulative Return</h3>
        <h2>{cumulative_return_simple:.4f}%</h2>
        <small>(End - Start) / Start</small>
    </div>
    """ , unsafe_allow_html=True)

with col6:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Avg Daily Return</h3>
        <h2>{avg_daily_return:.4f}%</h2>
        <small>Arithmetic mean</small>
    </div>
    """ , unsafe_allow_html=True)

with col7:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Daily Volatility</h3>
        <h2>{daily_volatility:.4f}%</h2>
        <small>Standard deviation</small>
    </div>
    """ , unsafe_allow_html=True)

with col8:
    st.markdown(f"""
    <div class='metric-card'>
        <h3>Max Drawdown</h3>
        <h2>{max_drawdown:.4f}%</h2>
        <small>Maximum loss from peak</small>
    </div>
    """ , unsafe_allow_html=True)

if show_daily_geometric:
    st.markdown("<h2 class='sub-header'>📁 Daily Geometric Returns from Excel</h2>", unsafe_allow_html=True)

```

```

daily_geometric_table = pd.DataFrame({
    'Date': geometric_data['Date'].dt.strftime('%Y-%m-%d'),
    'Portfolio (1+Return)': geometric_data['Portfolio_Geometric'].round(6),
    'Portfolio Cumul Return %': ((geometric_data['Portfolio_Geometric'] - 1) * 100).round(4),
    'TUNINDEX (1+Return)': geometric_data['TUNINDEX_Geometric'].round(6),
    'TUNINDEX Cumul Return %': ((geometric_data['TUNINDEX_Geometric'] - 1) * 100).round(4),
    'TUNINDEX Daily Return %': (geometric_data['TUNINDEX_Return'] * 100).round(4)
})

st.markdown('<div class="data-table">', unsafe_allow_html=True)
st.dataframe(daily_geometric_table, use_container_width=True)
st.markdown('</div>', unsafe_allow_html=True)

st.markdown("""
Table Explanation:
- Portfolio (1+Return): Cumulative product of (1 + daily return) from Excel Column J
- Portfolio Cumul Return %: Cumulative geometric return percentage = (1+Return - 1) * 100
- TUNINDEX (1+Return): Cumulative product of (1 + TUNINDEX return) from Excel Column P
- TUNINDEX Cumul Return %: Cumulative geometric return percentage for TUNINDEX
- TUNINDEX Daily Return %: Daily TUNINDEX returns from Excel Column N
""")

st.markdown("<h2 class='sub-header'>📊 Portfolio Value Over Time</h2>", unsafe_allow_html=True)

fig_value = go.Figure()
fig_value.add_trace(go.Scatter(
    x=portfolio_data['Date'],
    y=portfolio_data['Ending_Value'],
    mode='lines+markers',
    name='Portfolio Value',
    line=dict(color='#3B82F6', width=3),
    marker=dict(size=8),
    hovertemplate='Date: %{x|%Y-%m-%d}<br>Value: %{y:.2f} TND<extra></extra>'
))

fig_value.update_layout(
    title='Portfolio Value Trend',
    xaxis_title='Date',
    yaxis_title='Portfolio Value (TND)',
    height=400,
    template='plotly_white',
    xaxis=dict(tickformat='%Y-%m-%d', tickangle=45)
)
st.plotly_chart(fig_value, use_container_width=True)

```

```

st.markdown("<h2 class='sub-header'>📊 Geometric Returns Comparison</h2>", unsafe_allow_html=True)

fig_geometric = go.Figure()
fig_geometric.add_trace(go.Scatter(
    x=geometric_data['Date'],
    y=(geometric_data['Portfolio_Geometric'] - 1) * 100,
    mode='lines+markers',
    name='Portfolio',
    line=dict(color='#3B82F6', width=3),
    marker=dict(size=8),
    hovertemplate='Date: %{x|%Y-%m-%d}<br>Return: %{y:.4f}%<extra></extra>'
))

fig_geometric.add_trace(go.Scatter(
    x=geometric_data['Date'],
    y=(geometric_data['TUNINDEX_Geometric'] - 1) * 100,
    mode='lines+markers',
    name='TUNINDEX',
    line=dict(color='#10B981', width=3, dash='dash'),
    marker=dict(size=8),
    hovertemplate='Date: %{x|%Y-%m-%d}<br>Return: %{y:.4f}%<extra></extra>'
))

fig_geometric.update_layout(
    title='Cumulative Geometric Returns',
    xaxis_title='Date',
    yaxis_title='Cumulative Return (%)',
    height=400,
    template='plotly_white',
    xaxis=dict(tickformat='%Y-%m-%d', tickangle=45)
)

st.plotly_chart(fig_geometric, use_container_width=True)

if show_pie_chart:
    st.markdown("<h2 class='sub-header'>🥧 Interactive Portfolio Composition</h2>", unsafe_allow_html=True)

    col9, col10 = st.columns(2)

    with col9:

        pie_stocks = st.multiselect(
            "Select stocks for portfolio composition:",
            options=all_stocks,
            default=all_stocks,
            key="pie_stocks"
        )

```

```

with col10:

    available_dates = portfolio_data['Date'].dt.strftime('%Y-%m-%d').tolist()
    selected_date = st.selectbox(
        "Select date for portfolio composition:",
        options=available_dates,
        index=len(available_dates)-1,
        key="pie_date"
    )

selected_date_dt = pd.to_datetime(selected_date)

if pie_stocks:

    col11, col12 = st.columns([2, 1])

    with col11:

        pie_data = []
        colors = ['#3B82F6', '#10B981', '#EF4444', '#F59E0B', '#8B5CF6', '#EC4899', '#14B8A6']

        for i, stock in enumerate(pie_stocks):
            if stock in portfolio_weights:

                weight_df = portfolio_weights[stock]
                weight_row = weight_df[weight_df['Date'] == selected_date_dt]

                if not weight_row.empty:
                    weight = weight_row['Weight'].iloc[0] * 100
                    color_idx = i % len(colors)
                    pie_data.append({
                        'Stock': stock,
                        'Weight': weight,
                        'Color': colors[color_idx]
                    })

        if pie_data:

            labels = [item['Stock'] for item in pie_data]
            values = [item['Weight'] for item in pie_data]
            colors_list = [item['Color'] for item in pie_data]

            fig_pie = go.Figure(data=[go.Pie(
                labels=labels,
                values=values,
                hole=0.3,
                marker=dict(colors=colors_list),
                textinfo='label+percent',

```

```

        hovertemplate='<b>{%label}</b><br>Weight: {%value:.2f}%<br>Percentage: {%percent}<extra></extra>'
    ))

    fig_pie.update_layout(
        title=f"Portfolio Composition on {selected_date}",
        height=400,
        showlegend=False
    )

    st.plotly_chart(fig_pie, use_container_width=True)
else:
    st.warning(f"No weight data available for {selected_date}")

with col12:

    if pie_data:
        weight_df = pd.DataFrame({
            'Stock': [item['Stock'] for item in pie_data],
            'Weight (%)': [f"{item['Weight']:.2f}" for item in pie_data]
        })

        st.markdown("### Portfolio Weights")
        st.markdown('<div class="data-table">', unsafe_allow_html=True)
        st.dataframe(weight_df, use_container_width=True)
        st.markdown('</div>', unsafe_allow_html=True)

        total_weight = sum([item['Weight'] for item in pie_data])
        st.metric("Total Portfolio Weight", f"{total_weight:.2f}%")

if pie_stocks:
    st.markdown("### Portfolio Composition Over Time")

    fig_weights = go.Figure()

    for i, stock in enumerate(pie_stocks[:5]):
        if stock in portfolio_weights:
            weight_df = portfolio_weights[stock]
            color_idx = i % len(colors)
            fig_weights.add_trace(go.Scatter(
                x=weight_df['Date'],
                y=weight_df['Weight'] * 100,
                mode='lines+markers',
                name=stock,
                line=dict(color=colors[color_idx], width=2),
                marker=dict(size=6),
                hovertemplate=f'{stock}<br>Date: {%x|%Y-%m-%d}<br>Weight: {%y:.2f}%<extra></extra>'
            ))

```

```

    ))

fig_weights.update_layout(
    title='Portfolio Weight Evolution (Actual Weights from Excel)',
    xaxis_title='Date',
    yaxis_title='Weight (%)',
    height=400,
    template='plotly_white',
    xaxis=dict(tickformat='%Y-%m-%d', tickangle=45),
    hovermode='x unified'
)

st.plotly_chart(fig_weights, use_container_width=True)

if show_stock_performance and selected_stocks:
    st.markdown(f"<h2 class='sub-header'>📊 Selected Stocks Performance</h2>", unsafe_allow_html=True)

    tab1, tab2 = st.tabs(["Price Trends", "Performance Summary"])

    with tab1:
        fig_stocks = go.Figure()
        colors = ['#3B82F6', '#10B981', '#EF4444', '#F59E0B', '#8B5CF6', '#EC4899', '#14B8A6']

        for i, stock in enumerate(selected_stocks):
            if stock in stock_data and 'Close_Price' in stock_data[stock].columns:
                color_idx = i % len(colors)
                fig_stocks.add_trace(go.Scatter(
                    x=stock_data[stock]['Date'],
                    y=stock_data[stock]['Close_Price'],
                    mode='lines+markers',
                    name=stock,
                    line=dict(color=colors[color_idx], width=2),
                    marker=dict(size=6),
                    hovertemplate=f'{stock}<br>Date: %{{x}}%Y-%m-%d}<br>Price: %{{y:.2f}} TND<extra></extra>'
                ))

        fig_stocks.update_layout(
            title='Stock Price Trends',
            xaxis_title='Date',
            yaxis_title='Price (TND)',
            hovermode='x unified',
            height=400,
            template='plotly_white',
            showlegend=True,
            xaxis=dict(tickformat='%Y-%m-%d', tickangle=45)
        )

    st.plotly_chart(fig_stocks, use_container_width=True)

```

```

with tab2:

    summary_data = []

    for stock in selected_stocks:
        if stock in stock_data:
            stock_df = stock_data[stock]

            if len(stock_df) > 0:
                initial_price = stock_df['Close_Price'].iloc[0]
                final_price = stock_df['Close_Price'].iloc[-1]
                total_return = ((final_price - initial_price) / initial_price) * 100

                if 'Daily_Return' in stock_df.columns:
                    avg_return = stock_df['Daily_Return'].mean() * 100
                    volatility = stock_df['Daily_Return'].std() * 100
                else:
                    avg_return = 0
                    volatility = 0

                summary_data.append({
                    'Stock': stock,
                    'Start Price (TND)': f"{initial_price:.2f}",
                    'End Price (TND)': f"{final_price:.2f}",
                    'Total Return %': f"{total_return:.2f}",
                    'Avg Daily %': f"{avg_return:.2f}",
                    'Volatility %': f"{volatility:.2f}"
                })

            if summary_data:
                st.markdown('<div class="data-table">', unsafe_allow_html=True)
                summary_df = pd.DataFrame(summary_data)
                st.dataframe(summary_df, use_container_width=True)
                st.markdown('</div>', unsafe_allow_html=True)
            else:
                st.info("No stock data available for selected stocks")

st.markdown("<h2 class='sub-header'>📊 Summary Statistics</h2>", unsafe_allow_html=True)

col13, col14, col15, col16 = st.columns(4)

with col13:
    st.metric("Initial Value", f"{initial_value:.2f} TND")

with col14:
    st.metric("Final Value", f"{final_value:.2f} TND")

with col15:
    absolute_gain = final_value - initial_value

```

```

    st.metric("Absolute Gain", f"{absolute_gain:.2f} TND")

with col16:
    st.metric("Trading Days", len(portfolio_data))

st.markdown("---")
st.markdown("""
<div style='text-align: center; color: #6B7280; padding: 2rem;'>
    <p><strong>Portfolio Performance Dashboard</strong> • Using Exact Values from Excel</p>
    <p>Geometric Returns from Columns J & P • Actual Weights from Portfolio_Data_Corrected.xlsx</p>
</div>
""", unsafe_allow_html=True)

```

Technical Analysis:

```
# REQUIRED PACKAGES
# pip install pandas
# pip install numpy
# pip install plotly
# pip install openpyxl

import pandas as pd
import numpy as np
import re
from unicodedata import normalize as uni_normalize
import plotly.graph_objects as go
from datetime import datetime

def norm(s):
    if not isinstance(s, str):
        return s

    s2 = uni_normalize('NFKD', s).encode('ascii','ignore').decode('ascii')
    s2 = re.sub(r'\s+', ' ', s2).strip().lower()
    return s2

def find_in_sheet(df, patterns):
    norm_map = {c: norm(c) for c in df.columns}
    for c, n in norm_map.items():
        if any(p == n for p in patterns):
            return c
    return None

def get_date_col(df):
    return find_in_sheet(df, ['date', 'time', 'datetime'])

def parse_date_robust(date_series):
    try:
        return pd.to_datetime(date_series, errors='coerce')
    except:
        pass

    for fmt in ['%Y-%m-%d', '%d/%m/%Y', '%m/%d/%Y', '%Y.%m.%d', '%d-%m-%Y', '%d.%m.%Y']:
        try:
            return pd.to_datetime(date_series, format=fmt, errors='coerce')
        except:
            continue

    return pd.to_datetime(date_series, errors='coerce')
```

```

def clean_numeric_column(series):
    series_numeric = pd.to_numeric(series, errors='coerce')

    if series_numeric.isna().sum() > len(series) * 0.5:
        def extract_number(x):
            if isinstance(x, str):
                numbers = re.findall(r'[-+]?[d*]\.d+|\d+', x)
                if numbers:
                    return float(numbers[0])
            return x

        series = series.apply(extract_number)
        series_numeric = pd.to_numeric(series, errors='coerce')

    return series_numeric

xl_path = 'AMV.xlsx'
all_sheets = pd.read_excel(xl_path, sheet_name=None, engine='openpyxl')

print("\n=== DATE DIAGNOSTICS ===")
for sheet_name, df in all_sheets.items():
    date_col = get_date_col(df)
    if date_col:
        sample_dates = df[date_col].head(3).tolist()
        print(f"Sheet: {sheet_name}")
        print(f"Date column: '{date_col}'")
        print(f"Sample raw values: {sample_dates}")
        print(f>Data types of sample: {[type(d) for d in sample_dates]}")
        print(f"Unique types in column: {df[date_col].apply(type).unique()}")

        parsed = parse_date_robust(df[date_col].head(3))
        print(f"Parsed dates: {parsed.tolist()}")
        print("-" * 50)

FIELDS = {
    'Price':      ['cours ajuste'],
    'RSI':        ['14-day rsi'],
    'MACD':       ['macd line'],
    'MACD_Signal': ['signal line'],
    'MACD_Hist':  ['macd histogram'],
    'OBV':        ['obv'],
    'MA_5':       ['simple moving avg (5)'],
    'MA_10':      ['simple moving avg (10)'],
    'MA_20':      ['simple moving avg (20)'],
    'MA_50':      ['simple moving avg (50)'],

```

```

    'MA_200':      ['simple moving avg (200)'],
    'EMA_20':      ['exp moving avg (20)'],
    'EMA_50':      ['exp moving avg (50)'],
}

found = {k: None for k in FIELDS}
sheet_order = sorted(all_sheets.keys(), key=lambda s: 0 if 'scoring' in norm(s) else 1)

print("Scanning for columns...")
for sh in sheet_order:
    df = all_sheets[sh]
    date_col = get_date_col(df)
    if date_col is None:
        continue

    print(f" Sheet: {sh}")
    print(f"   Date column: {date_col}")

    for field, patterns in FIELDS.items():
        if found[field] is not None:
            continue

        col = find_in_sheet(df, patterns)
        if col:
            found[field] = (sh, date_col, col)
            print(f"   Found {field} at column: {col}")
            preview_vals = df[col].head(5).tolist()
            print(f"       Preview: {preview_vals}")

print("\n" + "="*50)
print("Summary of found columns:")
for field, info in found.items():
    if info:
        sh, date_col, value_col = info
        print(f" {field}: Sheet='{sh}', Date='{date_col}', Value='{value_col}'")
    else:
        print(f" {field}: NOT FOUND")

if found['Price'] is None:
    raise ValueError("Price column not found.")

anchor_sh, anchor_date, anchor_col = found['Price']
anchor_df = all_sheets[anchor_sh].copy()

anchor_df['Date'] = parse_date_robust(anchor_df[anchor_date])
anchor_df[anchor_col] = clean_numeric_column(anchor_df[anchor_col])
anchor_df = anchor_df[['Date', anchor_col]].dropna()

```

```

anchor_df.rename(columns={anchor_col: 'Price'}, inplace=True)

print(f"\nPrice data shape: {anchor_df.shape}")
print(f"Price date range: {anchor_df['Date'].min()} to {anchor_df['Date'].max()}")

master = (
    anchor_df
    .sort_values('Date')
    .drop_duplicates('Date')
    .set_index('Date')
)

for field, info in found.items():
    if field == 'Price' or info is None:
        continue

    sh, date_col, value_col = info
    temp = all_sheets[sh].copy()
    temp['Date'] = parse_date_robust(temp[date_col])

    temp[value_col] = clean_numeric_column(temp[value_col])

    if field.startswith('EMA_'):
        print(f"\n⚠ Special handling for {field}: Removing initial 0/NaN values")
        valid_mask = (temp[value_col].notna()) & (temp[value_col] != 0)
        if valid_mask.any():
            first_valid_idx = valid_mask[valid_mask].index[0]
            mask_before_valid = temp.index < first_valid_idx
            temp.loc[mask_before_valid, value_col] = np.nan
            print(f" First valid EMA value starts at index {first_valid_idx}")

        ema_values = temp[value_col].dropna()
        if len(ema_values) > 1:
            ema_array = ema_values.values
            if len(ema_array) > 5:
                first_5_values = ema_array[:5]
                diffs = np.diff(first_5_values)
                pct_changes = np.abs(diffs / first_5_values[:-1])

                stable_idx = np.where(pct_changes < 0.05)[0]
                if len(stable_idx) > 0:
                    stable_start = stable_idx[0] + 1
                    if stable_start > 0 and stable_start < len(ema_values):
                        stable_date = ema_values.index[stable_start]
                        mask_before_stable = temp.index < stable_date
                        temp.loc[mask_before_stable, value_col] = np.nan
                        print(f" Stable EMA starts at {stable_date}")

```

```

if field.startswith('MA_'):
    print(f"\n⚠ Special handling for {field}: Removing initial 0/NaN values")
    valid_mask = (temp[value_col].notna()) & (temp[value_col] != 0)
    if valid_mask.any():
        first_valid_idx = valid_mask[valid_mask].index[0]
        mask_before_valid = temp.index < first_valid_idx
        temp.loc[mask_before_valid, value_col] = np.nan
        print(f"    First valid MA value starts at index {first_valid_idx}")

temp = temp[['Date', value_col]].dropna()
temp.rename(columns={value_col: field}, inplace=True)

print(f"\nMerging {field}:")
print(f"    Shape before merge: {temp.shape}")
print(f"    Date range: {temp['Date'].min()} to {temp['Date'].max()}")
print(f"    Non-null values: {temp[field].notna().sum()}/{len(temp)}")
if len(temp) > 0:
    print(f"    First 3 values: {temp[field].head(3).tolist()}")
    print(f"    Last 3 values: {temp[field].tail(3).tolist()}")

master = master.merge(
    temp.set_index('Date'),
    left_index=True,
    right_index=True,
    how='left'
)

print(f"\nMaster DataFrame shape: {master.shape}")
print(f"Master DataFrame columns: {list(master.columns)}")
print(f"Master date range: {master.index.min()} to {master.index.max()}")

print("\n" + "="*50)
print("CHECKING FOR MISSING MOVING AVERAGES")
print("="*50)

missing_mas = []
for ma_field in ['MA_5', 'MA_10', 'MA_20', 'MA_50', 'MA_200']:
    if ma_field in master.columns and master[ma_field].notna().sum() > 0:
        non_null_pct = master[ma_field].notna().sum() / len(master) * 100
        print(f"✓ {ma_field}: Found in data ({non_null_pct:.1f}% non-null)")
    else:
        print(f"X {ma_field}: NOT FOUND in data")
        missing_mas.append(ma_field)

print("\n" + "="*50)
print("FINAL CLEANING OF MA AND EMA DATA")

```

```

print("="*50)

def clean_trend_column(series, column_name, is_ema=False):
    """Clean MA/EMA columns by removing initial NaN/zero values and ensuring smooth start"""
    if series is None or len(series) == 0:
        return series

    series_clean = series.copy()

    non_nan_mask = series_clean.notna()

    if non_nan_mask.any():
        first_non_nan_idx = series_clean[non_nan_mask].index[0]
        first_non_nan_val = series_clean.loc[first_non_nan_idx]

        print(f"\n  Cleaning {column_name}:")
        print(f"    First non-NaN value at {first_non_nan_idx.date()}: {first_non_nan_val}")

        if abs(first_non_nan_val) < 0.001:
            print(f"    Warning: First value is near zero ({first_non_nan_val})")
            reasonable_mask = (series_clean.notna()) & (abs(series_clean) > 0.001)
            if reasonable_mask.any():
                first_reasonable_idx = series_clean[reasonable_mask].index[0]
                mask_before_reasonable = series_clean.index < first_reasonable_idx
                series_clean.loc[mask_before_reasonable] = np.nan
                print(f"    Set values before {first_reasonable_idx.date()} to NaN")

        if is_ema and series_clean.notna().sum() > 5:
            ema_values = series_clean.dropna()
            ema_array = ema_values.values[:10]

            if len(ema_array) > 3:
                diffs = np.diff(ema_array[:5])
                if len(diffs) > 0:
                    pct_changes = np.abs(diffs / ema_array[:4])
                    large_jumps = pct_changes > 0.1
                    if large_jumps.any():
                        stable_mask = pct_changes < 0.05
                        if stable_mask.any():
                            first_stable = np.where(stable_mask)[0]
                            if len(first_stable) > 0:
                                stable_start = first_stable[0] + 1
                                if stable_start < len(ema_values):
                                    stable_date = ema_values.index[stable_start]
                                    mask_before_stable = series_clean.index < stable_date
                                    series_clean.loc[mask_before_stable] = np.nan
                                    print(f"    EMA stabilized at {stable_date.date()}")

```

```

    return series_clean

print("\nCleaning EMA columns:")
for ema_col in ['EMA_20', 'EMA_50']:
    if ema_col in master.columns:
        master[ema_col] = clean_trend_column(master[ema_col], ema_col, is_ema=True)

    ema_data = master[ema_col].dropna()
    if len(ema_data) > 0:
        print(f"    {ema_col}: {len(ema_data)} data points")
        print(f"        Starts at: {ema_data.index[0].date()} (value: {ema_data.iloc[0]:.2f})")
        print(f"        Ends at: {ema_data.index[-1].date()} (value: {ema_data.iloc[-1]:.2f})")

print("\nCleaning MA columns:")
for ma_col in ['MA_5', 'MA_10', 'MA_20', 'MA_50', 'MA_200']:
    if ma_col in master.columns:
        master[ma_col] = clean_trend_column(master[ma_col], ma_col, is_ema=False)

    ma_data = master[ma_col].dropna()
    if len(ma_data) > 0:
        print(f"    {ma_col}: {len(ma_data)} data points")
        print(f"        Starts at: {ma_data.index[0].date()} (value: {ma_data.iloc[0]:.2f})")

print("\n" + "="*50)
print("INTERPOLATING DATA")
print("="*50)

num_cols = master.select_dtypes(include='number').columns
for col in num_cols:
    non_null_pct = master[col].notna().sum() / len(master) * 100

    if col.startswith('EMA_'):
        print(f"Skipping interpolation for {col}: EMA should not be interpolated")
        continue

    if non_null_pct > 10:

        if col not in ['EMA_20', 'EMA_50', 'MA_200', 'MA_50']:
            master[col] = master[col].interpolate(limit_direction='both')
            print(f"Interpolated {col}: {non_null_pct:.1f}% non-null before interpolation")
        else:

```

```

        master[col] = master[col].interpolate(limit_direction='forward')

        print(f"Forward interpolated {col}: {non_null_pct:.1f}% non-null before interpolation")

fig_price = go.Figure()

price_trace = go.Scatter(
    x=master.index,
    y=master['Price'],
    mode='lines',
    name='Price',
    line=dict(color='#00BFFF', width=3)
)
fig_price.add_trace(price_trace)

ema_colors = {
    'EMA_20': '#FF4D4D',
    'EMA_50': '#00FF7F'
}

for ema_key, color in ema_colors.items():
    if ema_key in master.columns:

        ema_data = master[ema_key].dropna()
        if len(ema_data) > 0:
            trace = go.Scatter(
                x=ema_data.index,
                y=ema_data.values,
                mode='lines',
                name=ema_key.replace('_', ' '),
                line=dict(color=color, width=2),
                visible=True
            )
            fig_price.add_trace(trace)
            print(f"✅ Added {ema_key}: {len(ema_data)} data points (starting from {ema_data.index[0].date()})")

ema_buttons = []

all_visibility = [True] + [True] * (len(fig_price.data) - 1)
ema_buttons.append(
    dict(
        label="All EMAs",
        method="update",
        args=[{"visible": all_visibility}]
    )
)

```

```
)

price_only_visibility = [True] + [False] * (len(fig_price.data) - 1)
ema_buttons.append(
    dict(
        label="Price Only",
        method="update",
        args=[{"visible": price_only_visibility}]
    )
)
)
```

```
for i in range(1, len(fig_price.data)):
    trace = fig_price.data[i]
    visibility = [True] + [False] * (len(fig_price.data) - 1)
    visibility[i] = True

    ema_buttons.append(
        dict(
            label=f"Price + {trace.name}",
            method="update",
            args=[{"visible": visibility}]
        )
    )
)
```

```
fig_price.update_layout(
    title='Price + EMAs (Use dropdown to toggle EMAs)',
    plot_bgcolor='rgb(40,40,40)',
    paper_bgcolor='rgb(40,40,40)',
    font=dict(color='white'),
    xaxis=dict(
        rangelslider=dict(visible=True),
        type='date',
        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        title='Price',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=600,
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=ema_buttons,
```

```

        pad={"r": 10, "t": 10},
        showactive=True,
        x=0.02,
        xanchor="left",
        y=1.02,
        yanchor="bottom",
        bgcolor='rgba(0,0,0,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white')
    ),
]
)

fig_price.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_price.show()
print("\n✅ Price + EMAs chart created with dropdown toggle buttons (Price always shown)")

fig_ma = go.Figure()

price_trace_ma = go.Scatter(
    x=master.index,
    y=master['Price'],
    mode='lines',
    name='Price',
    line=dict(color='#00BFFF', width=3)

```

```

)
fig_ma.add_trace(price_trace_ma)

ma_colors = {
    'MA_5': '#FFD700',
    'MA_10': '#FF69B4',
    'MA_20': '#00FFFF',
    'MA_50': '#7CFC00',
    'MA_200': '#FFA500'
}

ma_traces_list = []

for ma_key, color in ma_colors.items():
    if ma_key in master.columns and master[ma_key].notna().sum() > 0:

        ma_data = master[ma_key].dropna()
        if len(ma_data) > 0:
            trace = go.Scatter(
                x=ma_data.index,
                y=ma_data.values,
                mode='lines',
                name=ma_key.replace('_', ' '),
                line=dict(color=color, width=2),
                visible=True,
                connectgaps=False
            )
            fig_ma.add_trace(trace)
            ma_traces_list.append(trace)
            print(f"✅ Added {ma_key}: {len(ma_data)} data points (starting from {ma_data.index[0].date()})")

print(f"\n✅ Added {len(ma_traces_list)} MA traces to the chart")

if len(ma_traces_list) == 0:
    print("⚠️ No MA data found for chart. Check if MA columns exist in your Excel file.")
else:

    ma_buttons = []

    all_ma_visibility = [True] + [True] * len(ma_traces_list)
    ma_buttons.append(
        dict(
            label="All MAs",
            method="update",
            args=[{"visible": all_ma_visibility}]
        )
    )

```

```

)

price_only_ma_visibility = [True] + [False] * len(ma_traces_list)
ma_buttons.append(
    dict(
        label="Price Only",
        method="update",
        args=[{"visible": price_only_ma_visibility}]
    )
)

```

```

for i in range(1, len(fig_ma.data)):
    trace = fig_ma.data[i]
    visibility = [True] + [False] * len(ma_traces_list)
    visibility[i] = True

    ma_buttons.append(
        dict(
            label=f"Price + {trace.name}",
            method="update",
            args=[{"visible": visibility}]
        )
    )

```

```

common_combinations = [
    ("Price + Short-term (5,10)", [1, 2]),
    ("Price + Medium-term (20,50)", [3, 4]),
    ("Price + Long-term (50,200)", [4, 5]),
    ("Price + All Short (5,10,20)", [1, 2, 3]),
]

```

```

for label, indices in common_combinations:
    if all(idx < len(fig_ma.data) for idx in indices):
        visibility = [True] + [False] * len(ma_traces_list)
        for idx in indices:
            if idx < len(visibility):
                visibility[idx] = True
        ma_buttons.append(
            dict(
                label=label,
                method="update",
                args=[{"visible": visibility}]
            )
        )

```

```

fig_ma.update_layout(

```

```

title=f'Price + Moving Averages (Use dropdown to toggle MAs)',
plot_bgcolor='rgb(40,40,40)',
paper_bgcolor='rgb(40,40,40)',
font=dict(color='white'),
xaxis=dict(
    rangelslider=dict(visible=True),
    type='date',
    title='Date',
    gridcolor='rgba(255,255,255,0.1)'
),
yaxis=dict(
    title='Value',
    gridcolor='rgba(255,255,255,0.1)'
),
hovermode='x unified',
showlegend=True,
height=600,
updatemenus=[
    dict(
        type="dropdown",
        direction="down",
        buttons=ma_buttons,
        pad={"r": 10, "t": 10},
        showactive=True,
        x=0.02,
        xanchor="left",
        y=1.02,
        yanchor="bottom",
        bgcolor='rgba(0,0,0,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white')
    ),
]
)

fig_ma.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',

```

```

        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_ma.show()

print("☑ Price + Moving Averages chart created with dropdown toggle buttons (Price always shown)")

if 'RSI' in master.columns:
    fig_rsi = go.Figure()
    rsi_data = master['RSI'].dropna()

    if len(rsi_data) > 0:

        rsi_trace = go.Scatter(
            x=rsi_data.index,
            y=rsi_data.values,
            mode='lines',
            name='RSI',
            line=dict(color='#FF3333', width=2)
        )
        fig_rsi.add_trace(rsi_trace)

        overbought_trace = go.Scatter(
            x=[rsi_data.index[0], rsi_data.index[-1]],
            y=[70, 70],
            mode='lines',
            name='Overbought (70)',
            line=dict(color='red', width=1, dash='dash'),
            opacity=0.5
        )

        oversold_trace = go.Scatter(
            x=[rsi_data.index[0], rsi_data.index[-1]],
            y=[30, 30],
            mode='lines',
            name='Oversold (30)',
            line=dict(color='green', width=1, dash='dash'),
            opacity=0.5
        )

        middle_trace = go.Scatter(
            x=[rsi_data.index[0], rsi_data.index[-1]],

```

```

        y=[50, 50],
        mode='lines',
        name='Middle (50)',
        line=dict(color='yellow', width=0.5, dash='dot'),
        opacity=0.3
    )

    fig_rsi.add_trace(overbought_trace)
    fig_rsi.add_trace(oversold_trace)
    fig_rsi.add_trace(middle_trace)

rsi_buttons = [
    dict(
        label="RSI + All Levels",
        method="update",
        args=[{"visible": [True, True, True, True]}]
    ),
    dict(
        label="RSI Only",
        method="update",
        args=[{"visible": [True, False, False, False]}]
    ),
    dict(
        label="RSI + Overbought/Oversold",
        method="update",
        args=[{"visible": [True, True, True, False]}]
    ),
]

fig_rsi.update_layout(
    title='RSI (Use dropdown to toggle levels)',
    plot_bgcolor='rgb(40,40,40)',
    paper_bgcolor='rgb(40,40,40)',
    font=dict(color='white'),
    xaxis=dict(
        rangeflider=dict(visible=True),
        type='date',
        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        range=[0,100],
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=500,
    updatemenus=[

```

```

        dict(
            type="dropdown",
            direction="down",
            buttons=rsi_buttons,
            pad={"r": 10, "t": 10},
            showactive=True,
            x=0.02,
            xanchor="left",
            y=1.02,
            yanchor="bottom",
            bgcolor='rgba(0,0,0,0.7)',
            bordercolor='white',
            borderwidth=1,
            font=dict(color='white')
        ),
    ]
)

fig_rsi.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_rsi.show()
print("☑ RSI chart created with dropdown toggle buttons")
else:
    print("⚠ RSI column exists but has no valid data")

fig_macd = go.Figure()

macd_traces = []
trace_names = []

```

```

if 'MACD' in master.columns:
    macd_data = master['MACD'].dropna()
    if len(macd_data) > 0:
        macd_trace = go.Scatter(
            x=macd_data.index,
            y=macd_data.values,
            mode='lines',
            name='MACD',
            line=dict(color='#00BFFF', width=2),
            visible=True
        )
        fig_macd.add_trace(macd_trace)
        macd_traces.append(macd_trace)
        trace_names.append('MACD')

if 'MACD_Signal' in master.columns:
    signal_data = master['MACD_Signal'].dropna()
    if len(signal_data) > 0:
        signal_trace = go.Scatter(
            x=signal_data.index,
            y=signal_data.values,
            mode='lines',
            name='Signal',
            line=dict(color='#FF69B4', width=2),
            visible=True
        )
        fig_macd.add_trace(signal_trace)
        macd_traces.append(signal_trace)
        trace_names.append('Signal')

if 'MACD_Hist' in master.columns:
    hist_data = master['MACD_Hist'].dropna()
    if len(hist_data) > 0:
        hist_trace = go.Bar(
            x=hist_data.index,
            y=hist_data.values,
            name='Histogram',
            marker_color=np.where(hist_data.values >= 0, '#00FF7F', '#FF4D4D'),
            opacity=0.7,
            visible=True
        )
        fig_macd.add_trace(hist_trace)
        macd_traces.append(hist_trace)
        trace_names.append('Histogram')

```

```

if len(macd_traces) > 0:

    macd_buttons = []

    macd_buttons.append(
        dict(
            label="All Components",
            method="update",
            args=[{"visible": [True] * len(fig_macd.data)}]
        )
    )

    for i, name in enumerate(trace_names):
        visibility = [False] * len(fig_macd.data)
        visibility[i] = True
        macd_buttons.append(
            dict(
                label=f"{name} Only",
                method="update",
                args=[{"visible": visibility}]
            )
        )

    if 'MACD' in trace_names and 'Signal' in trace_names:

        macd_idx = trace_names.index('MACD')
        signal_idx = trace_names.index('Signal')
        visibility = [False] * len(fig_macd.data)
        visibility[macd_idx] = True
        visibility[signal_idx] = True
        macd_buttons.append(
            dict(
                label="MACD + Signal",
                method="update",
                args=[{"visible": visibility}]
            )
        )

    fig_macd.update_layout(
        title='MACD (Use dropdown to toggle components)',
        plot_bgcolor='rgb(40,40,40)',
        paper_bgcolor='rgb(40,40,40)',
        font=dict(color='white'),
        xaxis=dict(
            rangelslider=dict(visible=True),
            type='date',

```

```

        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        title='MACD Value',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=600,
    bargap=0,
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=macd_buttons,
            pad={"r": 10, "t": 10},
            showactive=True,
            x=0.02,
            xanchor="left",
            y=1.02,
            yanchor="bottom",
            bgcolor='rgba(0,0,0,0.7)',
            bordercolor='white',
            borderwidth=1,
            font=dict(color='white')
        ),
    ]
)

fig_macd.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

```

```

    )

    )
    fig_macd.show()
    print("✅ MACD chart created with dropdown toggle buttons")
else:
    print("⚠️ No MACD data available for chart")

if 'OBV' in master.columns:
    fig_obv = go.Figure()
    obv_data = master['OBV'].dropna()

    if len(obv_data) > 0:

        obv_trace = go.Scatter(
            x=obv_data.index,
            y=obv_data.values,
            mode='lines',
            name='OBV',
            line=dict(color='#00E5FF', width=2)
        )
        fig_obv.add_trace(obv_trace)

    if len(obv_data) > 20:
        obv_ma_20 = obv_data.rolling(window=20).mean()
        obv_ma_trace = go.Scatter(
            x=obv_ma_20.index,
            y=obv_ma_20.values,
            mode='lines',
            name='OBV MA(20)',
            line=dict(color='#FF69B4', width=1, dash='dash'),
            opacity=0.7,
            visible=True
        )
        fig_obv.add_trace(obv_ma_trace)

    obv_buttons = [
        dict(
            label="OBV + MA",
            method="update",
            args=[{"visible": [True, True]}]
        ),
        dict(
            label="OBV Only",
            method="update",
            args=[{"visible": [True, False]}]
        ),

```

```

]

fig_obv.update_layout(
    title='On-Balance Volume (OBV)',
    plot_bgcolor='rgb(40,40,40)',
    paper_bgcolor='rgb(40,40,40)',
    font=dict(color='white'),
    xaxis=dict(
        rangeflider=dict(visible=True),
        type='date',
        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        title='OBV',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=500,
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=obv_buttons,
            pad={"r": 10, "t": 10},
            showactive=True,
            x=0.02,
            xanchor="left",
            y=1.02,
            yanchor="bottom",
            bgcolor='rgba(0,0,0,0.7)',
            bordercolor='white',
            borderwidth=1,
            font=dict(color='white')
        ),
    ]
)
else:
    fig_obv.update_layout(
        title='On-Balance Volume (OBV)',
        plot_bgcolor='rgb(40,40,40)',
        paper_bgcolor='rgb(40,40,40)',
        font=dict(color='white'),
        xaxis=dict(
            rangeflider=dict(visible=True),
            type='date',
            title='Date',
            gridcolor='rgba(255,255,255,0.1)'

```

```

    ),
    yaxis=dict(
        title='OBV',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=500
)

fig_obv.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_obv.show()

print("☑ OBV chart created with toggle button for moving average")
else:
    print("⚠ OBV column exists but has no valid data")

print("\n" + "="*50)
print("GOLDEN CROSS & DEATH CROSS ANALYSIS")
print("="*50)

if 'MA_50' in master.columns and 'MA_200' in master.columns:
    ma_50_data = master['MA_50'].dropna()
    ma_200_data = master['MA_200'].dropna()

    if len(ma_50_data) > 0 and len(ma_200_data) > 0:
        print("☑ Required MAs found for Golden/Death Cross analysis")

```

```

common_dates = ma_50_data.index.intersection(ma_200_data.index)
cross_data = master.loc[common_dates, ['Price', 'MA_50', 'MA_200']].copy()

cross_data['MA_50_above_MA_200'] = cross_data['MA_50'] > cross_data['MA_200']

cross_data['Prev_MA_50_above'] = cross_data['MA_50_above_MA_200'].shift(1)

cross_data['Golden_Cross'] = (
    (cross_data['Prev_MA_50_above'] == False) &
    (cross_data['MA_50_above_MA_200'] == True)
)

cross_data['Death_Cross'] = (
    (cross_data['Prev_MA_50_above'] == True) &
    (cross_data['MA_50_above_MA_200'] == False)
)

golden_cross_dates = cross_data[cross_data['Golden_Cross']].index
death_cross_dates = cross_data[cross_data['Death_Cross']].index

print(f"📌 Found {len(golden_cross_dates)} Golden Cross signals")
print(f"📌 Found {len(death_cross_dates)} Death Cross signals")

if len(golden_cross_dates) > 0:
    print("\nGolden Cross dates:")
    for date in golden_cross_dates[:10]:
        price_val = cross_data.loc[date, 'Price']
        ma50_val = cross_data.loc[date, 'MA_50']
        ma200_val = cross_data.loc[date, 'MA_200']
        print(f" {date.date()}: Price={price_val:.2f}, MA50={ma50_val:.2f}, MA200={ma200_val:.2f}")

if len(death_cross_dates) > 0:
    print("\nDeath Cross dates:")
    for date in death_cross_dates[:10]:
        price_val = cross_data.loc[date, 'Price']
        ma50_val = cross_data.loc[date, 'MA_50']
        ma200_val = cross_data.loc[date, 'MA_200']
        print(f" {date.date()}: Price={price_val:.2f}, MA50={ma50_val:.2f}, MA200={ma200_val:.2f}")

if len(golden_cross_dates) > 0:
    fig_golden = go.Figure()

```

```

fig_golden.add_trace(go.Scatter(
    x=cross_data.index,
    y=cross_data['Price'],
    mode='lines',
    name='Price',
    line=dict(color='#00BFFF', width=2)
))

fig_golden.add_trace(go.Scatter(
    x=cross_data.index,
    y=cross_data['MA_50'],
    mode='lines',
    name='MA_50',
    line=dict(color='#7CFC00', width=2)
))

fig_golden.add_trace(go.Scatter(
    x=cross_data.index,
    y=cross_data['MA_200'],
    mode='lines',
    name='MA_200',
    line=dict(color='#FFA500', width=2)
))

golden_prices = cross_data.loc[golden_cross_dates, 'Price']
fig_golden.add_trace(go.Scatter(
    x=golden_cross_dates,
    y=golden_prices * 0.98,
    mode='markers',
    name='Golden Cross',
    marker=dict(
        symbol='triangle-up',
        size=12,
        color='#00FF00',
        line=dict(width=2, color='white')
    ),
    hovertemplate='<b>Golden Cross</b><br>Date: %{x|%Y-%m-%d}<br>Price: %{y:.2f}<extra></extra>'
))

fig_golden.update_layout(
    title=f'Golden Cross Signals (MA_50 crosses above MA_200) - {len(golden_cross_dates)} signals',
    plot_bgcolor='rgb(40,40,40)',
    paper_bgcolor='rgb(40,40,40)',
    font=dict(color='white'),
    xaxis=dict(
        ranglider=dict(visible=True),

```

```

        type='date',
        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        title='Price',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=600
)

golden_buttons = [
    dict(
        label="Price + Both MAs",
        method="update",
        args=[{"visible": [True, True, True, True]}]
    ),
    dict(
        label="Price Only",
        method="update",
        args=[{"visible": [True, False, False, False]}]
    ),
    dict(
        label="Price + MA_50",
        method="update",
        args=[{"visible": [True, True, False, False]}]
    ),
    dict(
        label="Price + MA_200",
        method="update",
        args=[{"visible": [True, False, True, False]}]
    ),
    dict(
        label="MAS Only",
        method="update",
        args=[{"visible": [False, True, True, True]}]
    ),
]

fig_golden.update_layout(
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=golden_buttons,
            pad={"r": 10, "t": 10},

```

```

        showactive=True,
        x=0.02,
        xanchor="left",
        y=1.02,
        yanchor="bottom",
        bgcolor='rgba(0,0,0,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white')
    ),
]
)

fig_golden.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_golden.show()
print("\n☑ Golden Cross chart created")
else:
    print("⚠ No Golden Cross signals found to plot")

if len(death_cross_dates) > 0:
    fig_death = go.Figure()

    fig_death.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['Price'],
        mode='lines',

```

```

        name='Price',
        line=dict(color='#00BFFF', width=2)
    ))

    fig_death.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['MA_50'],
        mode='lines',
        name='MA_50',
        line=dict(color='#7CFC00', width=2)
    ))

    fig_death.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['MA_200'],
        mode='lines',
        name='MA_200',
        line=dict(color='#FFA500', width=2)
    ))

    death_prices = cross_data.loc[death_cross_dates, 'Price']
    fig_death.add_trace(go.Scatter(
        x=death_cross_dates,
        y=death_prices * 1.02,
        mode='markers',
        name='Death Cross',
        marker=dict(
            symbol='triangle-down',
            size=12,
            color='#FF0000',
            line=dict(width=2, color='white')
        ),
        hovertemplate='<b>Death Cross</b><br>Date: %{x|%Y-%m-%d}<br>Price: %{y:.2f}<extra></extra>'
    ))

    fig_death.update_layout(
        title=f'Death Cross Signals (MA_50 crosses below MA_200) - {len(death_cross_dates)} signals',
        plot_bgcolor='rgb(40,40,40)',
        paper_bgcolor='rgb(40,40,40)',
        font=dict(color='white'),
        xaxis=dict(
            rangelslider=dict(visible=True),
            type='date',
            title='Date',
            gridcolor='rgba(255,255,255,0.1)'
        ),
    ),

```

```

yaxis=dict(
    title='Price',
    gridcolor='rgba(255,255,255,0.1)'
),
hovermode='x unified',
showlegend=True,
height=600
)

death_buttons = [
    dict(
        label="Price + Both MAs",
        method="update",
        args=[{"visible": [True, True, True, True]}]
    ),
    dict(
        label="Price Only",
        method="update",
        args=[{"visible": [True, False, False, False]}]
    ),
    dict(
        label="Price + MA_50",
        method="update",
        args=[{"visible": [True, True, False, False]}]
    ),
    dict(
        label="Price + MA_200",
        method="update",
        args=[{"visible": [True, False, True, False]}]
    ),
    dict(
        label="MAs Only",
        method="update",
        args=[{"visible": [False, True, True, True]}]
    ),
]

fig_death.update_layout(
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=death_buttons,
            pad={"r": 10, "t": 10},
            showactive=True,
            x=0.02,
            xanchor="left",
            y=1.02,

```

```

        yanchor="bottom",
        bgcolor='rgba(0,0,0,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white')
    ),
]
)

fig_death.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

fig_death.show()
print("☑ Death Cross chart created")
else:
    print("⚠ No Death Cross signals found to plot")

if len(golden_cross_dates) > 0 or len(death_cross_dates) > 0:
    print("\n" + "="*50)
    print("CREATING COMBINED CROSSOVER CHART")
    print("="*50)

    fig_combined = go.Figure()

    fig_combined.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['Price'],
        mode='lines',

```

```

        name='Price',
        line=dict(color='FFFFFF', width=1.5),
        opacity=0.7
    ))

    fig_combined.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['MA_50'],
        mode='lines',
        name='MA_50 (Short-term)',
        line=dict(color='#7CFC00', width=2)
    ))

    fig_combined.add_trace(go.Scatter(
        x=cross_data.index,
        y=cross_data['MA_200'],
        mode='lines',
        name='MA_200 (Long-term)',
        line=dict(color='FFA500', width=2)
    ))

    if len(golden_cross_dates) > 0:
        golden_prices = cross_data.loc[golden_cross_dates, 'Price']
        fig_combined.add_trace(go.Scatter(
            x=golden_cross_dates,
            y=golden_prices * 0.98,
            mode='markers',
            name='Golden Cross',
            marker=dict(
                symbol='triangle-up',
                size=10,
                color='#00FF00',
                line=dict(width=1, color='white')
            ),
            hovertemplate='<b>Golden Cross</b><br>Date: %{x|%Y-%m-%d}<br>Price: %{y:.2f}<extra></extra>'
        ))

    if len(death_cross_dates) > 0:
        death_prices = cross_data.loc[death_cross_dates, 'Price']
        fig_combined.add_trace(go.Scatter(
            x=death_cross_dates,
            y=death_prices * 1.02,
            mode='markers',
            name='Death Cross',
            marker=dict(

```

```

        symbol='triangle-down',
        size=10,
        color='#FF0000',
        line=dict(width=1, color='white')
    ),
    hovertemplate='<b>Death Cross</b><br>Date: %{x|%Y-%m-%d}<br>Price: %{y:.2f}<extra></extra>'
))

cross_data['MA50_above_MA200'] = cross_data['MA_50'] > cross_data['MA_200']

above_periods = []
current_start = None

for i in range(len(cross_data)):
    if cross_data['MA50_above_MA200'].iloc[i]:
        if current_start is None:
            current_start = cross_data.index[i]
        else:
            if current_start is not None:
                above_periods.append((current_start, cross_data.index[i]))
                current_start = None

if current_start is not None:
    above_periods.append((current_start, cross_data.index[-1]))

for start_date, end_date in above_periods:
    fig_combined.add_vrect(
        x0=start_date, x1=end_date,
        fillcolor="rgba(0, 255, 0, 0.1)",
        layer="below", line_width=0,
        annotation_text="Golden Zone",
        annotation_position="top left",
        annotation_font_size=10,
        annotation_font_color="green"
    )

fig_combined.update_layout(
    title=f'Combined Crossover Analysis ({len(golden_cross_dates)} Golden, {len(death_cross_dates)}
Death)',
    plot_bgcolor='rgb(40,40,40)',
    paper_bgcolor='rgb(40,40,40)',
    font=dict(color='white'),
    xaxis=dict(
        rangelslider=dict(visible=True),
        type='date',

```

```

        title='Date',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    yaxis=dict(
        title='Price',
        gridcolor='rgba(255,255,255,0.1)'
    ),
    hovermode='x unified',
    showlegend=True,
    height=600,
    annotations=[
        dict(
            x=0.02, y=1.08,
            xref="paper", yref="paper",
            text="🟢 <b>Golden Cross</b>: MA_50 crosses above MA_200 (Bullish)",
            showarrow=False,
            font=dict(size=12, color="green"),
            align="left"
        ),
        dict(
            x=0.02, y=1.04,
            xref="paper", yref="paper",
            text="🔴 <b>Death Cross</b>: MA_50 crosses below MA_200 (Bearish)",
            showarrow=False,
            font=dict(size=12, color="red"),
            align="left"
        )
    ]
)

combined_buttons = [
    dict(
        label="All Signals",
        method="update",
        args=[{"visible": [True, True, True, True, True]}]
    ),
    dict(
        label="Price + MAs Only",
        method="update",
        args=[{"visible": [True, True, True, False, False]}]
    ),
    dict(
        label="Signals Only",
        method="update",
        args=[{"visible": [False, False, False, True, True]}]
    ),
    dict(
        label="Golden Cross Only",

```

```

        method="update",
        args=[{"visible": [True, True, True, True, False]}]
    ),
    dict(
        label="Death Cross Only",
        method="update",
        args=[{"visible": [True, True, True, False, True]}]
    ),
]

fig_combined.update_layout(
    updatemenus=[
        dict(
            type="dropdown",
            direction="down",
            buttons=combined_buttons,
            pad={"r": 10, "t": 10},
            showactive=True,
            x=0.02,
            xanchor="left",
            y=1.12,
            yanchor="bottom",
            bgcolor='rgba(0,0,0,0.7)',
            bordercolor='white',
            borderwidth=1,
            font=dict(color='white')
        ),
    ]
)

fig_combined.update_xaxes(
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month", stepmode="backward"),
            dict(count=6, label="6m", step="month", stepmode="backward"),
            dict(count=1, label="YTD", step="year", stepmode="todate"),
            dict(count=1, label="1y", step="year", stepmode="backward"),
            dict(step="all")
        ]),
        bgcolor='rgba(0,0,0,0.7)',
        activecolor='rgba(0,150,255,0.7)',
        bordercolor='white',
        borderwidth=1,
        font=dict(color='white'),
        x=0.75,
        xanchor="left",
        y=0.99,
        yanchor="bottom"
    )
)

```

```

    )
)

fig_combined.show()
print("✅ Combined Crossover chart created")

if len(golden_cross_dates) > 0 and len(death_cross_dates) > 0:
    print("\n" + "="*50)
    print("CROSSOVER PERFORMANCE ANALYSIS")
    print("="*50)

    performance_results = []

    for signal_date, signal_type in [(d, 'Golden') for d in golden_cross_dates] + [(d, 'Death') for d in
death_cross_dates]:

        later_signals = [d for d in golden_cross_dates if d > signal_date] + [d for d in
death_cross_dates if d > signal_date]

        if later_signals:
            end_date = min(later_signals)
            period_data = cross_data.loc[signal_date:end_date]

            if len(period_data) > 1:
                start_price = period_data['Price'].iloc[0]
                end_price = period_data['Price'].iloc[-1]
                price_change = ((end_price - start_price) / start_price) * 100

                performance_results.append({
                    'Signal Date': signal_date,
                    'Signal Type': signal_type,
                    'Start Price': start_price,
                    'End Price': end_price,
                    'Price Change %': price_change,
                    'Duration (days)': len(period_data)
                })

    if performance_results:
        perf_df = pd.DataFrame(performance_results)

        print("\n📊 Performance Summary:")
        print("-" * 70)

        golden_perf = perf_df[perf_df['Signal Type'] == 'Golden']
        if len(golden_perf) > 0:
            avg_golden_return = golden_perf['Price Change %'].mean()

```

```

        avg_golden_duration = golden_perf['Duration (days)'].mean()
        print(f"\nGolden Cross ({len(golden_perf)} periods):")
        print(f"    Average Return: {avg_golden_return:.2f}%")
        print(f"    Average Duration: {avg_golden_duration:.0f} days")

        best_golden = golden_perf.loc[golden_perf['Price Change %'].idxmax()]
        worst_golden = golden_perf.loc[golden_perf['Price Change %'].idxmin()]
        print(f"    Best: {best_golden['Price Change %']:.2f}% from {best_golden['Signal
Date'].date()}")

        print(f"    Worst: {worst_golden['Price Change %']:.2f}% from {worst_golden['Signal
Date'].date()}")

    death_perf = perf_df[perf_df['Signal Type'] == 'Death']
    if len(death_perf) > 0:
        avg_death_return = death_perf['Price Change %'].mean()
        avg_death_duration = death_perf['Duration (days)'].mean()
        print(f"\nDeath Cross ({len(death_perf)} periods):")
        print(f"    Average Return: {avg_death_return:.2f}%")
        print(f"    Average Duration: {avg_death_duration:.0f} days")

        best_death = death_perf.loc[death_perf['Price Change %'].idxmax()]
        worst_death = death_perf.loc[death_perf['Price Change %'].idxmin()]
        print(f"    Best: {best_death['Price Change %']:.2f}% from {best_death['Signal Date'].date()}")
        print(f"    Worst: {worst_death['Price Change %']:.2f}% from {worst_death['Signal
Date'].date()}")

    if len(golden_perf) > 0 and len(death_perf) > 0:
        print(f"\n📊 Comparison:")
        print(f"    Golden Cross vs Death Cross return difference: {avg_golden_return -
avg_death_return:.2f}%")
        print(f"    Golden Cross success rate: {(golden_perf['Price Change %'] >
0).sum()}/{len(golden_perf)} ({(golden_perf['Price Change %'] > 0).sum()/len(golden_perf)*100:.1f}%")
        print(f"    Death Cross success rate: {(death_perf['Price Change %'] <
0).sum()}/{len(death_perf)} ({(death_perf['Price Change %'] < 0).sum()/len(death_perf)*100:.1f}%")

        perf_df.to_excel('crossover_performance_analysis.xlsx', index=False)
        print(f"\n✅ Performance analysis saved to 'crossover_performance_analysis.xlsx'")
    else:
        print("❌ Cannot create Golden/Death Cross charts: MA_50 or MA_200 columns exist but have no valid data")
else:
    print("❌ Cannot create Golden/Death Cross charts: Missing MA_50 or MA_200 data in Excel file")

master.to_excel('multisheet_indicators.xlsx', engine='openpyxl')

```

```

print(f"\n✅ Merged data saved to 'multisheet_indicators.xlsx'")
print(f"Columns: {list(master.columns)}")
print(f"Shape: {master.shape}")
print(f>Date range: {master.index.min()} to {master.index.max()}")

print("\n" + "="*50)
print("COMPLETE TECHNICAL ANALYSIS DASHBOARD")
print("="*50)
print("📊 ALL INTERACTIVE CHARTS CREATED:")
charts_created = []

if 'EMA_20' in master.columns or 'EMA_50' in master.columns:
    charts_created.append("1. Price + EMAs")
if any(ma in master.columns for ma in ['MA_5', 'MA_10', 'MA_20', 'MA_50', 'MA_200']):
    charts_created.append("2. Price + Moving Averages")
if 'RSI' in master.columns:
    charts_created.append("3. RSI")
if any(col in master.columns for col in ['MACD', 'MACD_Signal', 'MACD_Hist']):
    charts_created.append("4. MACD")
if 'OBV' in master.columns:
    charts_created.append("5. OBV")
if 'MA_50' in master.columns and 'MA_200' in master.columns:
    charts_created.append("6-8. Golden Cross & Death Cross Analysis")

for chart in charts_created:
    print(chart)

print("\n✅ KEY FEATURES:")
print("✓ EMA data cleaned - starts at first valid value (no initial jump)")
print("✓ MA data cleaned - starts at first valid value (no initial jump)")
print("✓ Special handling for EMA stability (removes volatile initial values)")
print("✓ Price always shown in EMA and MA charts")
print("✓ Interactive toggle buttons for all charts")
print("✓ Time range controls (1m, 6m, YTD, 1y, All)")
print("✓ Golden Cross detection (MA_50 > MA_200)")
print("✓ Death Cross detection (MA_50 < MA_200)")
print("✓ Performance analysis for crossovers")
print("✓ Dark theme for all charts")
print("✓ Data saved to Excel for further analysis")
print("\n📖 HOW TO USE:")
print("✓ LEFT SIDE: Dropdown to toggle indicators")
print("✓ RIGHT SIDE: Time buttons to zoom (1m, 6m, YTD, 1y, All)")
print("✓ BOTTOM: Range slider for precise zooming")
print("✓ HOVER: Over lines for detailed values")
print("✓ CLICK: Legend items to manually show/hide")
print("✓ DOUBLE-CLICK: Reset zoom")
print("\n🎯 DASHBOARD READY FOR TECHNICAL ANALYSIS!")

```

Sources:

<https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Note%20commext11mois%202024.pdf>

[https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Croissance Economique T3 2024.pdf](https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Croissance_Economique_T3_2024.pdf)

<https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/bms%20Mai%202024.pdf>

[https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Communiqu%C3%A9%20de%20presse IPC base 2015 Novembre 2024.pdf](https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Communiqu%C3%A9%20de%20presse_IPC_base_2015_Novembre_2024.pdf)

[https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Note Emploi T3 2024.pdf](https://www.ins.tn/sites/default/files-ftp3/files/publication/pdf/Note_Emploi_T3_2024.pdf)

<https://www.bct.gov.tn>

<https://www.tunisieclearing.com/tc/fr>

<https://www.investing.com/markets/tunisia>

<https://www.bvmt.com.tn>